



The Author's Experience, Strategy & Shortcuts

Disclaimer: This is a study and practice resource. It contains practice exercises and training materials designed to help prepare for the CKA exam. It does not include, share, or reproduce actual CKA exam questions. All exercises are independently designed training scenarios. See [CONTRIBUTING.md](#) and [CODE_OF_CONDUCT.md](#) for policies.

My CKA study notes, practice questions, and kubectl cheat sheet. Kubernetes v1.35. I scored 89% — this is everything I used to prepare.

CKA Certification Guide 2026 — How I Passed with 89%

I took the CKA in March 2026 and scored 89%. Writing this while it's fresh — partly because I was frustrated with how many outdated guides are still floating around (dockershim references in 2026, come on) and partly because organizing my notes helped me retain what I learned.

The [CKA](#) is a hands-on, terminal-based exam. 2 hours, roughly 17-25 tasks, no multiple choice. I prepped for about 4 weeks. This repo has my notes, the commands I actually used, YAML I wrote from memory, and the mistakes I made along the way.

Blog version of these notes: [Pass the CKA Certification Exam](#)

If this was useful, a star helps others find it.

Quick Start (< 4 Weeks to Exam)

If you're time-pressured, here's the fast track:

1. **Start a practice cluster** (*if needed*) — if `kubectl` says connection refused or you don't have a local cluster yet: `bash scripts/init-cluster.sh`. This creates a kind-based k8s 1.35.1 cluster with one command. [Read more](#).
2. **Run the setup script** — get your aliases and vim config right from day one: [scripts/exam-setup.sh](#)
3. **Do the exercises** — work through the [31 hands-on exercises](#) in order. Each one targets a specific CKA domain.
4. **Use YAML templates** — reference [TEMPLATES.md](#) for all skeleton YAML. Copy, paste, modify.
5. **Do the mock exam** — practice under exam conditions with timed scenarios. See [mock-exams/README.md](#) for prep strategy and scoring.
6. **Do killer.sh twice** — once 2 weeks out, once 3 days before. See [killer.sh vs the Real Exam](#).
7. **Read the exam day strategy** — the [two-pass approach](#) saved time on exam day.

Repo Structure

```
CKA-Certified-Kubernetes-Administrator/
├── README.md                      # This guide (you're here)
├── exercises/                    # 31 hands-on labs
│   ├── 01-pod-basics/
│   ├── 02-multi-container-pod/
│   ├── 03-configmap-secret/
│   ├── 04-rbac/
│   ├── 05-networkpolicy/
│   ├── 06-deployment-rollout/
│   ├── 07-statefulset/
│   ├── 08-node-drain-cordon/
│   ├── 09-kubeadm-upgrade/
│   ├── 10-static-pod/
│   ├── 11-troubleshoot-cluster/
│   ├── 12-storage-pv-pvc/
│   ├── 13-helm-install-upgrade/
│   ├── 14-kustomize-overlays/
│   ├── 15-gateway-api/
│   ├── 16-hpa/
│   ├── 17-kubectrl-debug/
│   ├── 18-cri-dockerd-setup/
│   ├── 19-ingress-classic/
│   ├── 20-pod-security-standards/
│   ├── 21-jobs-cronjobs/
│   ├── 22-priorityclass/
│   ├── 23-resource-requests-tuning/
│   ├── 24-priorityclass-patch/
│   ├── 25-storage-waitforfirstconsumer/
│   ├── 26-cri-dockerd-setup/
│   ├── 27-cni-tigera-install/
│   ├── 28-network-policy-complex/
│   ├── 29-troubleshoot-etcd-endpoint/
│   ├── 30-tls-configuration-update/
│   └── 31-argocd-gitops-setup/
├── TEMPLATES.md                  # All YAML templates in collapsible format
├── skeletons/                    # 23 YAML template files (see TEMPLATES.md)
├── mock-exams/                  # Full practice exams (15 questions, 2 hours each)
│   ├── MOCK-EXAM-01.md          # Practice questions
│   ├── MOCK-EXAM-01-SOLUTIONS.md # Complete solutions and explanations
│   ├── MOCK-EXAM-02.md
│   └── MOCK-EXAM-02-SOLUTIONS.md
├── cheatsheet/
│   └── cka-cheatsheet.md        # One-page printable reference
├── troubleshooting/
│   └── README.md                # Symptom-based lookup playbook
├── scripts/
│   ├── exam-setup.sh           # Aliases, vim config, bash completion
│   └── validate-local.sh       # Local YAML validation (run before pushing)
├── .github/
│   ├── workflows/validate.yml  # CI - YAML lint on every push
│   ├── ISSUE_TEMPLATE/
│   │   └── config.yml          # Bug, content request, exam feedback
│   └── PULL_REQUEST_TEMPLATE.md # Discussions link for questions
├── CHANGELOG.md
├── CODE_OF_CONDUCT.md
├── CONTRIBUTING.md
├── SECURITY.md
└── LICENSE
```

Table of Contents

Start Here — Practice Your Skills

- [CKA Syllabus Breakdown \(v1.35\)](#)
- [Domain 1 — Storage \(10%\)](#)
- [Domain 2 — Troubleshooting \(30%\)](#)
- [Domain 3 — Workloads & Scheduling \(15%\)](#)
- [Domain 4 — Cluster Architecture, Installation & Configuration \(25%\)](#)
- [Domain 5 — Services & Networking \(20%\)](#)

- [CKA Domain Weight Distribution](#)
- [Practice Scenarios with Full Solutions](#)
- [Mock Exams — Final Preparation](#)
- [Study Progress Tracker](#)

Exam Preparation Strategy

- [The Exam Environment \(PSI Remote Desktop\)](#)
- [First 60 Seconds — Aliases, vim, bash](#)
- [Imperative Commands Quick Reference](#)
- [YAML Templates Quick Reference](#)
- [Exam Day Strategy — Time Allocation](#)
- [Mistakes That Will Fail You on the CKA](#)
- [Vim Keys I Actually Used on Exam Day](#)
- [Troubleshooting Decision Flowchart](#)
- [CKA Exam Day Checklist](#)

Study Resources & Planning

- [kubectl Cheat Sheet for CKA](#)
- [Docs Pages I Actually Used During the Exam](#)
- [CKA Study Plan \(4-5 Weeks\)](#)
- [Study Resources for CKA 2026](#)
- [killer.sh vs the Real CKA Exam](#)

Exam Details & Reference

- [CKA Exam Details — Cost, Duration, Passing Score, Format](#)
- [How Much Does the CKA Exam Cost?](#)
- [CKA vs CKAD vs CKS — Which One Should You Take?](#)
- [CKA vs CKAD vs CKS Scope Architecture Diagram](#)
- [What Changed in Kubernetes v1.35 for CKA](#)
- [Before You Book the CKA Exam](#)
- [CKA FAQ — Common Questions](#)

Final Thoughts

- [Final Words](#)

Local Cluster Setup — Practice with Kind

Don't have a Kubernetes cluster? Use the included script to bootstrap a local k8s 1.35.1 cluster via [kind](#) (Kubernetes in Docker). Perfect for running through all 31 exercises without needing a kubeadm setup.

Quick Start: One Command

```
bash scripts/init-cluster.sh
```

What it does: - Detects your OS and architecture (x86_64, aarch64, etc.) - Auto-installs `kind` if missing (v0.31.0) - Validates Docker is running - Creates a named cluster `cka-practice` with k8s 1.35.1 - Configures kubeconfig automatically - Prints next steps on success

Detailed Setup (if needed manually)

Prerequisites: - [Docker](#) — installed and running - `kubectl` — installed (via [official docs](#) or `brew install kubectl`) - ~5 GB free disk space - ~2GB free RAM (kind cluster runs in that)

Script Options:

```
# Create a new cluster (or use existing)
bash scripts/init-cluster.sh

# Recreate the cluster (wipe and start fresh)
bash scripts/init-cluster.sh --recreate

# Show help
bash scripts/init-cluster.sh --help
```

After setup: 1. Run the exam-setup script to configure aliases: `bash scripts/exam-setup.sh` 2. Verify connection: `kubectl get nodes` — should show one node `cka-practice-control-plane` 3. Start an exercise: `cd exercises/01-pod-basics && cat README.md`

Using Your Own Cluster

If you already have: - A kubeadm cluster (bare metal / VMs) - An existing kind cluster - A cloud cluster (EKS, GKE, AKS) — **not recommended for CKA practice** (different tooling)

Then skip init-cluster.sh and just run:

```
bash scripts/exam-setup.sh      # Configure aliases, etc.
kubectl get nodes               # Verify connection
cd exercises/01-pod-basics      # Start practicing
```

CKA Exam Details — Cost, Duration, Passing Score, Format (March 2026)

CKA Exam Details	Information
Exam Type	Performance-based (live terminal — NOT multiple choice)
Exam Duration	2 hours
Passing Score	66%
Kubernetes Version	v1.35
Number of Questions	~17-25 tasks (varies per session)
Exam Cost	\$445 USD (includes one free retake)
Certificate Validity	2 years
Exam Delivery	PSI Secure Browser (remote proctored)
Allowed Resources	kubernetes.io/docs, kubernetes.io/blog, github.com/kubernetes — open in exam browser
Domains Covered	5 domains: Storage, Troubleshooting, Workloads & Scheduling, Cluster Architecture, Services & Networking
Exam Language	English, Japanese, Simplified Chinese
OS in Exam	Ubuntu Linux terminal

Important: the passing score is 66%, not 75% like some older guides say. They lowered it. Still not easy though — 2 hours goes fast when you're troubleshooting a broken kubelet under pressure.

[Back to top](#)

How Much Does the CKA Exam Cost?

The CKA costs **\$445 USD** as of March 2026. That includes:

- One exam attempt
- One free retake (if you fail)
- Two killer.sh simulator sessions (24 hours each)
- Access to a self-paced training course

Discount tips: - The CNCF runs sales on Black Friday and KubeCon weeks — I've seen 30-40% off - Linux Foundation bundles (CKA + CKAD) sometimes drop to ~\$500 total - Check if your employer has a training budget — most do for certs - Student discounts exist through the Linux Foundation

Don't pay full price if you can wait for a sale. I paid around \$300 during a KubeCon promo.

CKA vs CKAD vs CKS — Which One Should You Take?

	CKA	CKAD	CKS
Focus	Cluster administration	Application development	Security
Who it's for	SREs, platform engineers, admins	Developers deploying to K8s	Security engineers, senior admins
Difficulty	Hard — the troubleshooting and etcd questions are brutal under time pressure	Medium — if you already deploy to K8s, most of this is familiar	Hardest of the three — Falco and AppArmor syntax is miserable to memorize

	CKA	CKAD	CKS
Duration	2 hours	2 hours	2 hours
Passing Score	66%	66%	67%
Cost	\$445	\$445	\$445
Prerequisites	None	None	Must hold active CKA
Key Topics	etcd, kubeadm, RBAC, troubleshooting, networking	Pods, Deployments, Jobs, probes, volumes	Falco, AppArmor, OPA, Network Policies, audit
Questions	~17-25	~15-20	~15-20
My honest take	Start here if you manage clusters. The etcd and kubeadm skills don't exist anywhere else.	Easier than CKA but less impressive on a resume.	Skip unless your job requires it — the ROI is lower.

My take: if you're doing any kind of cluster administration, start with CKA. If you're purely a dev who deploys apps, CKAD first. CKS requires an active CKA, so you can't skip it.

There's about 40% overlap between CKA and CKAD (pods, deployments, services, configmaps, secrets). If you pass one, the other is easier. I did CKA first because troubleshooting and etcd backup are harder to learn on your own.

CKA vs CKAD vs CKS Scope Architecture Diagram

```
graph TB
    subgraph CKA["CKA - Cluster Administration"]
        style CKA fill:#326CE5,color:#fff
        A1[etcd backup/restore]
        A2[kubeadm install/upgrade]
        A3[RBAC - Roles, ClusterRoles]
        A4[Node management - drain, cordon]
        A5[Troubleshooting - kubelet, kube-proxy, CoreDNS]
        A6[Cluster networking - CNI, Services]
        A7[Storage - PV, PVC, StorageClass]
    end

    subgraph CKAD["CKAD - Application Development"]
        style CKAD fill:#00A86B,color:#fff
        B1[Multi-container pods - sidecars, init]
        B2[Jobs, CronJobs]
        B3[Probes - liveness, readiness, startup]
        B4[Helm charts]
        B5[Custom Resource Definitions]
        B6[Blue/green, canary deployments]
    end

    subgraph SHARED["Shared (~40% overlap)"]
        style SHARED fill:#FF8C00,color:#fff
        S1[Pods, Deployments, Services]
        S2[ConfigMaps, Secrets]
        S3[NetworkPolicies]
        S4[Ingress / Gateway API]
        S5[Labels, selectors, annotations]
        S6[Resource requests/limits]
    end

    subgraph CKS["CKS - Security"]
        style CKS fill:#DC143C,color:#fff
        C1[Falco runtime security]
        C2[AppArmor / Seccomp profiles]
        C3[OPA Gatekeeper]
        C4[Audit logging]
        C5[Image scanning - Trivy]
        C6[Pod Security Standards]
        C7[Supply chain security]
    end

    CKA -->|"~40% overlap"| CKAD
    CKA -->|"required for"| CKS
```

What Changed in Kubernetes v1.35 for CKA

If you're studying from a guide written for v1.29 or v1.30, some of it is wrong. I found this out the hard way — half my bookmarked blog posts had outdated sidecar syntax and still referenced `--record` on rollouts. Here's what actually changed that matters for the CKA:

Feature	Status in v1.35	CKA Impact
Sidecar containers (native)	GA	Init containers with <code>restartPolicy: Always</code> run as sidecars. You'll see this on the exam.
In-place pod vertical scaling	Beta	Can resize CPU/memory without restarting. I spent 30 minutes learning this and it wasn't on my exam. Know it exists, move on.
Gateway API	GA (v1.2+)	Replacing Ingress long-term. I got a question on this. Know how to create a Gateway and an HTTPRoute that points to a backend service.
cgroup v2	Default	All nodes use cgroup v2 now. Affects resource monitoring and limits.
kubectl debug	GA	<code>k debug node/<name></code> and <code>k debug pod/<name></code> — useful for troubleshooting tasks.
ValidatingAdmissionPolicy	GA	CEL-based admission without webhooks. I didn't get this on my exam but it's in the curriculum now. Worth 15 minutes of study.
Pod Scheduling Readiness	GA	Pods can wait in scheduling gates. Not likely to show up on the exam.
CSI migration complete	Done	In-tree volume plugins fully migrated. StorageClass provisioners are all CSI now.

The big ones for exam prep: native sidecars and Gateway API. If your study material doesn't cover these, it's outdated.

Before You Book the CKA Exam

Checklist I wish someone had given me before I started booking:

1. **Can you set up a cluster from scratch with kubeadm?** I couldn't the first time I tried. Took me 3 attempts before I could do it without the docs open. Do it at least twice before booking.
2. **Can you do an etcd backup and restore?** This is almost guaranteed to show up. I practiced this 10+ times. The cert flags need to be muscle memory, not something you look up.
3. **Are you comfortable with RBAC?** Role vs ClusterRole, RoleBinding vs ClusterRoleBinding, ServiceAccounts — I fumbled the `--as=system:serviceaccount:ns:name` syntax for weeks before it clicked.
4. **Can you troubleshoot a NotReady node?** SSH in, check kubelet, check certificates, check networking. This is 30% of the score and it's the section where most people lose the most time.
5. **Do you have a cluster to practice on?** kind or minikube on your laptop, or Killercoda/KodeKloud online. You cannot pass this exam by reading — you have to break things.
6. **Have you done killer.sh at least once?** killer.sh is good practice but don't assume the real exam is easier. Both have tricky questions. Killer.sh builds speed and confidence, which matters more. My first killer.sh score was terrible. That's normal.
7. **Is your ID ready?** Government-issued ID, matching your CNCF account name. Check this before exam day.

The Exam Environment (PSI Remote Desktop)

The exam runs in a PSI Secure Browser — a remote Ubuntu desktop. Things that surprised me:

Copy/Paste: - `Ctrl+Shift+C` / `Ctrl+Shift+V` in the terminal - Right-click paste works sometimes, sometimes it doesn't - The built-in notepad uses normal `Ctrl+C` / `Ctrl+V` - Practice these shortcuts. I wasted 2 minutes fumbling with paste in the first question.

Terminal quirks: - There's a small delay on every keystroke — maybe 50-100ms. It adds up. - Tab completion works but feels laggy. - You can open multiple terminal tabs. I used two: one for the task, one for verification. - The file browser is basic. Stick to command line.

Browser: - One extra tab allowed for kubernetes.io documentation - Bookmarks are not available — you'll type URLs manually - The search on kubernetes.io is your best friend. Use it instead of navigating.

General: - Webcam and mic are on the entire time - Clear your desk — nothing on it except your computer - No second monitor - No headphones/earphones - Water bottle is fine (clear, no label) - Bathroom breaks are allowed but the timer doesn't pause

Important: Exam Environment vs Practice

In the exam, each question is a fresh SSH connection. Any aliases or configuration you set up will NOT carry over to the next question. Do not waste exam time on setup.

Only reliable shortcut: The `k` alias for `kubectl` is pre-configured on every machine.

For practice on your laptop, you can use the setup script in `scripts/exam-setup.sh` to speed up drilling. But practice without these shortcuts 1-2 weeks before the exam to build command muscle memory. You need to know: - `kubectl run ...` - `kubectl create ...` - `--dry-run=client -o yaml` (type it out, not an alias) - `--force --grace-period=0` (type it out, not an alias)

Memorize the commands. That beats any alias on test day.

Imperative Commands Quick Reference

Why imperative? Under exam pressure (2 hours, ~17 tasks), typing YAML is slow. The CKA expects speed. Most questions can be solved faster with imperative commands than writing manifests. Declarative is for when you need complex control or for learning.

Speed tip: Memorize these command patterns. On test day, `kubectl run`, `kubectl create`, and `kubectl expose` will be your fastest friends.

Fast Pod Creation (15-30 seconds vs 2 minutes for YAML)

```
# Basic pod
k run nginx --image=nginx:1.27

# Pod with port exposed
k run nginx --image=nginx:1.27 --port=80

# Pod with labels
k run nginx --image=nginx:1.27 --labels=app=web,tier=frontend

# There is no direct command to run pod with resource limits, instead you can use dry-run then edit file or use --overrides
arg
k run nginx --image=nginx --overrides='{ "spec": { "containers": [ { "name": "nginx", "image": "nginx", "resources": { "requests": { "cpu": "100m", "memory": "128Mi" }, "limits": { "cpu": "200m", "memory": "256Mi" } } } ] } }'

# Pod with environment variables
k run nginx --image=nginx:1.27 --env=LOG_LEVEL=debug --env=APP_ENV=prod

# Pod with command override
k run nginx --image=nginx:1.27 -- sh -c "echo 'Hello' && sleep 3600"

# Pod with multiple containers (init + app)
k run myapp --image=myapp:1.0 --overrides='{ "spec": { "initContainers": [ { "name": "init", "image": "busybox", "command": [ "wget", "-O", "/data/file", "http://example.com" ] }, { "name": "myapp", "image": "myapp:1.0", "volumeMounts": [ { "name": "data", "mountPath": "/data" } ] }, "volumes": [ { "name": "data", "emptyDir": {} } ] } }'

# Generate YAML without running (for review/editing)
k run nginx --image=nginx:1.27 $do > pod.yaml
```

Exam pattern: Use `$do` flag to generate YAML, review it, then apply. Saves you from memorizing exact YAML structure.

Deployment Creation & Management (Most exam questions)

```
# Basic deployment
k create deployment webapp --image=nginx:1.27

# Deployment with replicas
k create deployment webapp --image=nginx:1.27 --replicas=3

# Deployment with resource requests
k create deployment webapp --image=nginx:1.27 --replicas=3 --dry-run=client -o yaml | \
  sed 's/resources: {}/resources:\n      requests:\n      cpu: 100m\n      memory: 128Mi/' >
  deploy.yaml

# Scale deployment
k scale deployment webapp --replicas=5

# Update image (rolling update)
k set image deployment/webapp nginx=nginx:1.28 --record

# Check rollout status
k rollout status deployment/webapp

# View rollout history
k rollout history deployment/webapp

# Rollback to previous version
k rollout undo deployment/webapp

# Rollback to specific revision
k rollout undo deployment/webapp --to-revision=2

# Pause rollout (for manual canary)
k rollout pause deployment/webapp

# Resume rollout
k rollout resume deployment/webapp

# Generate deployment YAML
k create deployment webapp --image=nginx:1.27 --dry-run=client -o yaml > deploy.yaml
```

Exam tip: Rollout commands appear on almost every CKA exam. Practice `rollout undo` and `rollout history` until they're muscle memory.

Service Exposure (ClusterIP, NodePort, LoadBalancer)

```
# Expose deployment as ClusterIP (default, internal only)
k expose deployment webapp --port=80 --target-port=8080

# Expose as NodePort (accessible on all nodes)
k expose deployment webapp --port=80 --target-port=8080 --type=NodePort

# Expose as LoadBalancer (cloud-only)
k expose deployment webapp --port=80 --target-port=8080 --type=LoadBalancer

# Get service external IP (NodePort/LoadBalancer)
k get svc -w

# Expose a pod directly (not recommended but does work)
k expose pod nginx --port=80 --name=web-svc

# Create service without deploying (generate YAML)
k create service clusterip web --tcp=80:8080 $do > svc.yaml
k create service nodeport web --tcp=80:8080 $do > svc.yaml

# Edit service after creation
k edit svc webapp

# Port forward for testing (like accessing the pod locally)
k port-forward svc/webapp 8080:80
```

Exam pattern: Most questions ask: "Expose deployment X on port Y." Use `k expose deployment X --port=Y --target-port=<app-port>`.

RBAC — Roles, ServiceAccounts, RoleBindings (25% of exam)

```
# Create ServiceAccount
k create sa my-app -n prod

# Create Role (allow specific verbs on specific resources)
k create role pod-reader --verb=get,list,watch --resource=pods -n prod
k create role pod-deleter --verb=get,list,delete --resource=pods -n prod

# Create RoleBinding (bind role to user/sa)
k create rolebinding read-pods --role=pod-reader --serviceaccount=prod:my-app -n prod

# ClusterRole (cross-namespace)
k create clusterrole node-reader --verb=get,list --resource=nodes

# ClusterRoleBinding
k create clusterrolebinding read-nodes --clusterrole=node-reader --serviceaccount=prod:my-app

# Check if user/SA has permission
k auth can-i list pods -n prod --as=system:serviceaccount:prod:my-app

# Check your own permissions
k auth can-i list pods -n prod

# View role details
k get role pod-reader -n prod -o yaml
k get rolebinding read-pods -n prod -o yaml

# Edit role to add/remove permissions
k edit role pod-reader -n prod
```

Exam tip: RBAC questions usually involve creating SA + Role + RoleBinding, then testing with `k auth can-i`. Practice the syntax until you don't have to think.

ConfigMaps & Secrets (Application config)

```
# ConfigMap from literal values
k create configmap app-config --from-literal=LOG_LEVEL=debug --from-literal=DB_HOST=postgres.prod

# ConfigMap from file
k create configmap app-config --from-file=config.properties

# ConfigMap from directory
k create configmap app-config --from-file=./configs/

# Secret from literal
k create secret generic db-secret --from-literal=username=admin --from-literal=password=secret123

# Secret from file
k create secret generic tls-secret --from-file=tls.crt=cert.pem --from-file=tls.key=key.pem

# Docker registry secret (for pulling private images)
k create secret docker-registry dockerhub --docker-server=docker.io --docker-username=myuser --docker-password=mypass

# View secret (NOT decrypted)
k get secret db-secret -o yaml

# Describe configmap
k describe cm app-config
```

Exam pattern: When a question mentions "app needs config from file," use `k create configmap $name --from-file`.

Node Management (Maintenance, upgrades, troubleshooting)

```
# Cordon node (mark unschedulable, don't evict existing pods)
k cordon node-1

# Drain node (evict all pods before maintenance)
k drain node-1 --ignore-daemonsets --delete-emptydir-data

# Uncordon node (resume scheduling)
k uncordon node-1

# Label a node
k label nodes node-1 disk=ssd
k label nodes node-1 disk=ssd --overwrite # update existing

# Taint a node (prevent pods from scheduling)
k taint nodes node-1 key=value:NoSchedule
k taint nodes node-1 key=value:NoExecute # evict existing pods

# Remove taint
k taint nodes node-1 key-

# Get node info (CPU, memory, conditions)
k describe node node-1

# Check node status
k get nodes -o wide
```

Exam pattern: "Prepare node for maintenance" = `k drain`. "Node is full" = label it and use nodeSelector. "Node needs maintenance" = `k cordon` + `k drain`.

Debugging & Troubleshooting (30% of exam — learn this well)

```
# Get pod logs (follow in real-time)
k logs pod-name
k logs pod-name -f
k logs pod-name --tail=50

# Logs from previous crashed pod
k logs pod-name --previous

# Logs from all containers in pod
k logs pod-name --all-containers

# Logs from specific container in multi-container pod
k logs pod-name -c container-name

# Short-lived troubleshooting — exec into pod
k exec -it pod-name -- /bin/bash
k exec -it pod-name -c container-name -- /bin/bash

# One-off command in pod
k exec pod-name -- curl http://localhost:8080

# Describe pod (events, conditions, resource usage)
k describe pod pod-name

# Describe everything about a resource
k describe node node-1

# Watch events in real-time
k get events -w

# Get specific event from a namespace
k get events -n prod --sort-by='.lastTimestamp'

# Port forward to debug (useful when service isn't working)
k port-forward pod-name 8080:8080
k port-forward svc/service-name 8080:8080

# Copy files from pod to local (for log inspection)
k cp pod-name:/var/log/app.log ./app.log

# Check resource metrics (requires metrics-server)
k top nodes
k top pods -n prod
```

Exam tip: 30% of exam is "troubleshoot why this isn't working." `k describe` and `k logs` are your debugging weapons. Learn to read the error messages.

Resource Quotas & Limits (Cluster resource management)

```
# Create LimitRange (per-pod limits)
k create limitrange cpu-limit --max=2 --min=100m --type=Pod

# Resource quota (per-namespace total limits)
k create quota my-quota --hard=requests.cpu=10,limits.cpu=20,requests.memory=100Gi,pods=100

# Check current usage
k describe resourcequota my-quota -n prod
k describe limitrange cpu-limit -n prod
```

Exam pattern: Usually appears as "create resource quota so namespace doesn't exceed X CPU."

Shortcuts & Pro Tips (Save 5-10 minutes per exam)

```
# Dry-run + output to file (review before applying)
k create deployment app --image=app:1.0 $do > deploy.yaml
k apply -f deploy.yaml

# Delete resources fast
k delete pod pod-name $now          # force deletion
k delete pods --all -n prod --now    # delete all pods in namespace
k delete deployment webapp -n prod   # cascade delete (pods too)

# Get resources in all namespaces
k get pods -A
k get pods --all-namespaces

# Get in custom columns (useful for spotting issues)
k get pods -o wide                   # show node, IP, etc.
k get pods -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image

# JSONPath queries (find pods by image)
k get pods -o jsonpath='{.items[*].metadata.name}' | xargs -I{} echo {}

# Get yaml for an existing resource then copy it
k get deployment webapp -o yaml > webapp-backup.yaml
k apply -f webapp-backup.yaml

# Edit resource live
k edit deployment webapp

# Patch resource (update specific field)
k patch deployment webapp -p '{"spec":{"replicas":5}}'
```

Docs Pages I Actually Used During the Exam

You can access kubernetes.io during the exam. These are the pages I remember opening — there were probably others I clicked through but these are the ones I went back to:

Topic	Page
kubectl cheat sheet	https://kubernetes.io/docs/reference/kubectl/cheatsheet/
etcd backup/restore	https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/
kubeadm upgrade	https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/
RBAC	https://kubernetes.io/docs/reference/access-authn-authz/rbac/
NetworkPolicy	https://kubernetes.io/docs/concepts/services-networking/network-policies/
PV / PVC	https://kubernetes.io/docs/concepts/storage/persistent-volumes/
Static pods	https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/
Taints and tolerations	https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/
Debug services	https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/
CoreDNS	https://kubernetes.io/docs/tasks/administer-cluster/coredns/
Ingress	https://kubernetes.io/docs/concepts/services-networking/ingress/
Gateway API	https://kubernetes.io/docs/concepts/services-networking/gateway/
Drain a node	https://kubernetes.io/docs/tasks/administer-cluster/safely-drain-node/

Tip: use the search bar on kubernetes.io. Don't waste time clicking through navigation menus.

kubectl Cheat Sheet for CKA

These are the commands I used most during the exam. All using the aliases from the setup section.

Context and Namespace

```
# Switch context (DO THIS BEFORE EVERY QUESTION)
k config use-context <context-name>

# Set default namespace
kn <namespace>

# Check current context
k config current-context
```

Pods

```
# Create a pod
k run nginx --image=nginx:1.27

# Create pod YAML without running it
k run nginx --image=nginx:1.27 $do > pod.yaml

# Pod with labels
k run nginx --image=nginx:1.27 --labels=app=web,tier=frontend

# Pod with port
k run nginx --image=nginx:1.27 --port=80

# Get pods with extra info
k get pods -o wide
k get pods --show-labels
k get pods -l app=web

# Delete pod fast
k delete pod nginx $now
```

Deployments

```
# Create deployment
k create deployment webapp --image=nginx:1.27 --replicas=3

# Generate YAML
k create deployment webapp --image=nginx:1.27 --replicas=3 $do > deploy.yaml

# Scale
k scale deployment webapp --replicas=5

# Update image
k set image deployment/webapp nginx=nginx:1.28

# Rollout commands
k rollout status deployment/webapp
k rollout history deployment/webapp
k rollout undo deployment/webapp
k rollout undo deployment/webapp --to-revision=2
```

Services

```
# Expose a deployment
k expose deployment webapp --port=80 --target-port=80 --type=ClusterIP
k expose deployment webapp --port=80 --target-port=80 --type=NodePort

# Expose a pod
k expose pod nginx --port=80 --name=nginx-svc

# Generate service YAML
k create service clusterip my-svc --tcp=80:80 $do > svc.yaml
```

RBAC

```
# Create ServiceAccount
k create sa my-sa -n my-ns

# Create Role
k create role pod-reader --verb=get,list,watch --resource=pods -n my-ns

# Create RoleBinding
k create rolebinding read-pods --role=pod-reader --serviceaccount=my-ns:my-sa -n my-ns

# Create ClusterRole
k create clusterrole node-reader --verb=get,list --resource=nodes

# Create ClusterRoleBinding
k create clusterrolebinding read-nodes --clusterrole=node-reader --serviceaccount=my-ns:my-sa

# Check permissions
k auth can-i list pods -n my-ns --as=system:serviceaccount:my-ns:my-sa
```

Node Management

```
# Cordon (mark unschedulable)
k cordon <node-name>

# Drain (evict pods)
k drain <node-name> --ignore-daemonsets --delete-emptydir-data

# Uncordon
k uncordon <node-name>

# Label a node
k label node <node-name> disk=ssd

# Taint a node
k taint nodes <node-name> key=value:NoSchedule

# Remove a taint
k taint nodes <node-name> key=value:NoSchedule-
```

etcd

```
# Snapshot
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify snapshot
ETCDCTL_API=3 etcdctl snapshot status /tmp/etcd-backup.db --write-table

# Restore
ETCDCTL_API=3 etcdctl snapshot restore /tmp/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored
```

Troubleshooting

```
# Node issues
k get nodes
k describe node <node-name>
ssh <node> -- sudo systemctl status kubelet
ssh <node> -- sudo journalctl -u kubelet --no-pager | tail -30

# Pod issues
k describe pod <pod-name>
k logs <pod-name>
k logs <pod-name> -c <container-name>
k logs <pod-name> --previous

# Service/endpoint issues
k get endpoints <service-name>
k get svc
k describe svc <service-name>

# DNS
k run test-dns --image=busybox:1.36 --rm -it -- nslookup kubernetes
k get pods -n kube-system -l k8s-app=kube-dns

# Debug node
k debug node/<node-name> -it --image=busybox:1.36
```

Quick YAML Generation

```
# Pod
k run nginx --image=nginx:1.27 $do > pod.yaml

# Deployment
k create deployment webapp --image=nginx:1.27 $do > deploy.yaml

# Service
k expose deployment webapp --port=80 $do > svc.yaml

# Job
k create job my-job --image=busybox:1.36 -- sh -c "echo done" $do > job.yaml

# CronJob
k create cronjob my-cron --image=busybox:1.36 --schedule="*/5 * * * *" -- sh -c "echo tick" $do > cron.yaml

# ConfigMap
k create configmap my-cm --from-literal=key=value $do > cm.yaml

# Secret
k create secret generic my-secret --from-literal=pass=s3cret $do > secret.yaml
```

[Back to top](#)

CKA Syllabus Breakdown (v1.35)

Domain 1 — Storage (10%)

10% of the score. Sounds small, but the questions are straightforward if you understand PV/PVC binding. I almost skipped this in my study plan and then it showed up as one of the easiest points on the exam. The main trap: `storageClassName` has to match exactly between PV and PVC, and "exactly" includes the case where one side has it set and the other doesn't.

See also: [Exercise 12 — Storage](#) / Skeletons: [pv.yaml](#), [pvc.yaml](#), [storageclass.yaml](#)

1.1 — Understand Storage Classes and Persistent Volumes

Storage on Kubernetes is simple in concept but annoying in practice. A **PV** is the actual storage (think: the hard drive). A **PVC** is a request for that storage (think: "I need 2Gi of disk"). A **StorageClass** tells Kubernetes how to dynamically create PVs when a PVC asks for one.

What actually matters for the exam: - PV is cluster-scoped (no namespace). PVC is namespace-scoped. I mixed these up and created a PVC in the wrong namespace — it bound fine but the pod couldn't see it. - PVC binds to a PV when: capacity >= request, accessModes match, AND storageClassName matches. If any one of these is off, the PVC sits in `Pending` forever with no helpful error message. - The `storageClassName` trap is real. `manual` ≠ `Manual` ≠ empty string. Triple-check it.

```
# PV — cluster-scoped
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/my-pv
```

```
# PVC — namespace-scoped
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
  storageClassName: manual
```

```
# Pod using PVC
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: my-pvc
```

1.2 — Understand Volume Mode, Access Modes, and Reclaim Policies

Access Modes — you need to know the four-letter abbreviations:

Mode	Short	What it actually means
ReadWriteOnce	RWO	One node can mount read-write. This is what you'll use 90% of the time.
ReadOnlyMany	ROX	Many nodes can mount read-only. Rarely comes up on the exam.
ReadWriteMany	RWX	Many nodes can mount read-write. Doesn't work with hostPath — I tried.
ReadWriteOncePod	RWOP	Only one pod can mount read-write. New in v1.29+, might show up.

Reclaim Policies — know the difference or you'll lose data:

Policy	What actually happens
Retain	PV survives PVC deletion. Data is safe but you have to manually clean up the PV before it can be reused.
Delete	PV and the underlying storage get nuked. This is the default for most cloud StorageClasses. Be careful.
Recycle	Deprecated. Don't use it, don't memorize it.

Volume Modes: - `Filesystem` (default) — mounted as a directory - `Block` — raw block device, no filesystem

On the exam: know the difference between Retain and Delete. If the question says "data should persist after PVC deletion," use Retain.

1.3 — Configure Applications with Persistent Storage

The pattern is always the same: 1. Create PV (or let StorageClass provision it dynamically) 2. Create PVC referencing the StorageClass 3. Mount PVC in the Pod spec

```
# StorageClass for dynamic provisioning
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/no-provisioner
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
```

`WaitForFirstConsumer` delays binding until a pod actually needs the volume. This avoids scheduling issues where the PV is on node A but the pod lands on node B.

Common gotcha: forgetting `storageClassName`. If the PVC has `storageClassName: ""` (empty string), it only binds to PVs with no StorageClass. If you set `storageClassName: manual`, the PV must also have `storageClassName: manual`.

Domain 2 — Troubleshooting (30%)

This is the biggest domain — 30% of the score. You'll get several questions asking you to fix broken things. This is where I lost the most time in early practice because I had no system. I'd randomly check pods, then nodes, then pods again. Once I built a consistent troubleshooting order (nodes → kubelet → control plane pods → describe → logs → endpoints), my accuracy went way up.

See also: [Exercise 11 — Troubleshoot Cluster](#) / [Troubleshooting Decision Flowchart](#)

2.1 — Evaluate Cluster and Node Logging

Where to find logs:

Component	Log Location
kubelet	<code>journalctl -u kubelet</code> (systemd service)
kube-apiserver	<code>/var/log/kube-apiserver.log</code> or <code>k logs -n kube-system kube-apiserver-<node></code>
kube-scheduler	<code>k logs -n kube-system kube-scheduler-<node></code>
kube-controller-manager	<code>k logs -n kube-system kube-controller-manager-<node></code>
etcd	<code>k logs -n kube-system etcd-<node></code>
Container runtime	<code>journalctl -u containerd</code>

Control plane components run as static pods (in `/etc/kubernetes/manifests/`), so you can check their logs with `k logs`. But kubelet is a systemd service — use `journalctl`.

```
# Check node status
k get nodes
k describe node <node-name>

# Check kubelet on a node
ssh <node>
sudo systemctl status kubelet
sudo journalctl -u kubelet --no-pager | tail -50

# Check control plane pods
k get pods -n kube-system
```

2.2 — Monitor Applications

Honestly, monitoring during the exam boils down to two things: `k describe` and `k get events`. I never used `k top` during my actual exam — metrics-server wasn't available on every cluster. But know it exists in case they ask.

```
# These two are 90% of exam monitoring
k describe pod <pod-name>      # events section at bottom tells you everything
k get events --sort-by='.lastTimestamp'

# The rest — know them, probably won't need them
k get pods
k top pods
k top nodes
k get events -n <namespace> --field-selector reason=Failed
```

2.3 — Container stdout/stderr Logs

The two you'll actually use on the exam: `k logs <pod>` and `k logs <pod> --previous`. That's it. The rest are nice-to-know but I never needed `-f` or `--tail` under exam time pressure.

```
# The essentials
k logs <pod-name>
k logs <pod-name> --previous          # crashed container – you'll use this a lot
k logs <pod-name> -c <container-name> # multi-container pods

# Rarely needed on the exam but useful
k logs <pod-name> -f
k logs <pod-name> --tail=50
k logs -l app=web --all-containers
```

2.4 — Troubleshoot Application Failure

The exam gives you a broken pod and you figure out why. After enough practice, you develop a reflex based on the status:

Pending — this is the most common one on the exam. 9 times out of 10 it's one of these: - PVC not bound → `k get pvc` — check `storageClassName` matches - Taint with no toleration → `k describe node` and look at Taints - No resources available → `k describe pod` events will say "Insufficient cpu" - Node selector or affinity doesn't match any node → check labels - I once spent 5 minutes on a Pending pod that just needed a namespace with a ResourceQuota increased

CrashLoopBackOff — the container starts and dies immediately: - `k logs <pod> --previous` first. Always. The error is usually obvious. - Wrong command/entrypoint is the sneaky one — I got tricked by `["sh", "-c"]` vs `["sh -c"]` once - Missing config (env vars, configmaps, secrets)

ImagePullBackOff — almost always a typo in the image name. Seriously. Check the image string character by character. - Private registry without `imagePullSecrets` is the other cause, but rare on the exam

Error / Failed: - `k describe pod` events section → `k logs` → you'll find it

```
# Systematic pod debugging
k get pod <pod> -o wide          # which node? what IP?
k describe pod <pod>             # events, conditions
k logs <pod>                     # app logs
k logs <pod> --previous          # if it crashed
k exec <pod> -- cat /etc/resolv.conf # DNS config
k exec <pod> -- env              # env vars loaded?
```

2.5 — Troubleshoot Cluster Component Failure

When the whole cluster is broken:

```
# 1. Are nodes ready?
k get nodes

# 2. Are control plane pods running?
k get pods -n kube-system

# 3. Is kubelet running on the node?
ssh <node>
sudo systemctl status kubelet
sudo systemctl restart kubelet # try restarting

# 4. Check kubelet logs
sudo journalctl -u kubelet --no-pager | tail -50

# 5. Are certificates expired?
sudo kubeadm certs check-expiration

# 6. Is etcd healthy?
ETCDCTL_API=3 etcdctl endpoint health \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# 7. Check static pod manifests
ls /etc/kubernetes/manifests/
# Should have: etcd.yaml, kube-apiserver.yaml, kube-controller-manager.yaml, kube-scheduler.yaml
```

Common causes: - kubelet not running → `systemctl start kubelet` - Wrong static pod manifest → fix YAML in `/etc/kubernetes/manifests/` - Certificates expired → `kubeadm certs renew all` - etcd data directory wrong → check `--data-dir` in etcd manifest - kube-apiserver flag wrong → check manifest, fix, wait for restart

2.6 — Troubleshoot Networking

Service not reachable? Work through this:

```
# 1. Does the service exist and have the right selector?
k get svc <service>
k describe svc <service>

# 2. Does the service have endpoints?
k get endpoints <service>
# If empty: selector doesn't match any running pod

# 3. Is the pod actually running on the target port?
k exec <pod> -- wget -q0- localhost:<port>

# 4. DNS working?
k run test-dns --image=busybox:1.36 --rm -it -- nslookup <service-name>

# 5. Is kube-proxy running?
k get pods -n kube-system -l k8s-app=kube-proxy

# 6. NetworkPolicy blocking traffic?
k get networkpolicy -n <namespace>
```

The most common networking issues on the exam: - Service selector doesn't match pod labels (typo in labels) - Service targetPort doesn't match container port - NetworkPolicy denying traffic (remember: any policy = deny by default for that pod) - CoreDNS down or misconfigured - kube-proxy not running on a node

Domain 3 — Workloads & Scheduling (15%)

15% of the score. Deployments, rolling updates, ConfigMaps, Secrets, static pods, scheduling constraints. I found this the most comfortable domain because it's what you do day-to-day. The gotcha: static pods. I kept trying to delete them with kubectl and wondering why they came back. Once you understand kubelet manages them directly, it clicks.

See also: [Exercise 01 — Pod Basics](#) / [Exercise 06 — Deployment Rollout](#) / [Exercise 10 — Static Pod](#)

3.1 — Understand Deployments and How to Perform Rolling Updates and Rollbacks

Deployments manage ReplicaSets, which manage Pods. When you update the image, Kubernetes creates a new ReplicaSet and gradually shifts pods over. The thing that tripped me up: the container name in `k set image` is the container name from the pod spec, not the deployment name. I kept writing `k set image deployment/webapp webapp=nginx:1.27` when the container was actually called `nginx`. Wasted 3 minutes every time.

```
# Create
k create deployment webapp --image=nginx:1.26 --replicas=3

# Update image (triggers rolling update)
k set image deployment/webapp nginx=nginx:1.27

# Watch the rollout
k rollout status deployment/webapp

# Check history
k rollout history deployment/webapp

# Rollback to previous
k rollout undo deployment/webapp

# Rollback to specific revision
k rollout undo deployment/webapp --to-revision=2
```

Rolling update strategy options:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # max pods above desired count during update
      maxUnavailable: 0 # max pods that can be unavailable during update
```

- `maxSurge: 1, maxUnavailable: 0` = zero-downtime (one extra pod at a time)
- `maxSurge: 0, maxUnavailable: 1` = no extra pods, one goes down at a time
- `Recreate` strategy = kill all old pods first, then create new ones (causes downtime)

3.2 — Use ConfigMaps and Secrets to Configure Applications

ConfigMaps hold non-sensitive config. Secrets hold sensitive data (base64-encoded, not encrypted by default).

```
# Create ConfigMap
k create configmap app-config \
  --from-literal=APP_MODE=production \
  --from-literal=LOG_LEVEL=info

# Create Secret
k create secret generic db-creds \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASS=changeme

# From file
k create configmap nginx-conf --from-file=nginx.conf
```

Three ways to inject into a pod:

1. Environment variables (all keys):

```
envFrom:
- configMapRef:
  name: app-config
- secretRef:
  name: db-creds
```

2. Single key as env var:

```
env:
- name: DATABASE_USER
  valueFrom:
    secretKeyRef:
      name: db-creds
      key: DB_USER
```

3. Mounted as files:

```
volumeMounts:
- name: config-vol
  mountPath: /etc/config
volumes:
- name: config-vol
  configMap:
    name: app-config
```

Gotcha: if you mount a ConfigMap as a volume at a directory, it replaces the entire directory. Use `subPath` to mount a single file without replacing the directory.

3.3 — Know How to Scale Applications

```
# Manual scaling
k scale deployment webapp --replicas=5

# Autoscaling (HPA — not heavily tested on CKA but know it exists)
k autoscale deployment webapp --min=2 --max=10 --cpu-percent=80
```

3.4 — Self-Healing Workloads: Deployments, DaemonSets, StatefulSets

The one you'll actually use on the exam is **Deployment**. It manages ReplicaSets, which manage Pods. You create Deployments, you scale them, you update them, you roll them back. That's 90% of this topic.

- **ReplicaSet**: Keeps N pods running. You almost never create these directly — Deployments create them for you.
- **Deployment**: This is the workhorse. Rolling updates, rollbacks, scaling. Know this cold.
- **DaemonSet**: One pod per node. Logging agents, monitoring. Comes up occasionally on the exam. The YAML is basically a Deployment without `replicas`.
- **StatefulSet**: Stable network identity and persistent storage. Barely on the CKA — know it exists, maybe know the headless service pattern, don't spend hours on it.

```
# DaemonSet – runs on every node
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-agent
spec:
  selector:
    matchLabels:
      app: log-agent
  template:
    metadata:
      labels:
        app: log-agent
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: agent
          image: fluentd:v1.17
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

3.5 — Understand How Resource Limits Can Affect Pod Scheduling

```
resources:
  requests:
    memory: "64Mi"    # scheduler uses this to find a node
    cpu: "250m"       # 250 millicores = 0.25 CPU
  limits:
    memory: "128Mi"   # OOMKilled if exceeded
    cpu: "500m"       # throttled if exceeded
```

- **Requests** = what the scheduler looks at when placing the pod. If no node has enough, pod stays Pending.
- **Limits** = ceiling. Memory over limit = OOMKilled. CPU over limit = throttled.
- If you set limits without requests, requests default to limits.
- LimitRange sets defaults and constraints for a namespace. ResourceQuota caps total usage.

3.6 — Awareness of Manifest Management and Common Templating Tools

On the CKA, you mostly write raw YAML. But know these exist: - **Kustomize**: `k apply -k <dir>` — built into kubectl, overlays and patches - **Helm**: package manager for Kubernetes — CKA may ask you to install a chart - **kubectl \$do**: generate YAML with `--dry-run=client -o yaml` and edit it

3.7 — Schedule Pods on Specific Nodes

nodeSelector (simplest):

```
spec:
  nodeSelector:
    disk: ssd
```

Node affinity (more flexible):

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disk
                operator: In
                values:
                  - ssd
```

Taints and tolerations:

Taints go on nodes. Tolerations go on pods.

```
# Taint a node
k taint nodes node1 gpu=true:NoSchedule

# Remove taint
k taint nodes node1 gpu=true:NoSchedule-
```

```
# Pod toleration
spec:
  tolerations:
  - key: "gpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
```

Effects: - `NoSchedule` — don't schedule new pods (existing stay) - `PreferNoSchedule` — try to avoid, but not strict - `NoExecute` — evict existing pods too

3.8 — Static Pods

Static pods are managed by the kubelet directly, not the API server. The kubelet watches a directory (usually `/etc/kubernetes/manifests/`) and creates pods from any YAML files it finds there.

```
# Find the static pod path
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# Usually: /etc/kubernetes/manifests

# Create a static pod
sudo tee /etc/kubernetes/manifests/static-web.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: static-web
spec:
  containers:
  - name: web
    image: nginx:1.27
    ports:
    - containerPort: 80
EOF
```

Static pods show up in `kubectl get pods` with the node name appended (e.g., `static-web-node1`). You can't delete them via `kubectl` — the kubelet recreates them. To remove: delete the manifest file.

Control plane components (kube-apiserver, kube-scheduler, kube-controller-manager, etcd) are all static pods.

Domain 4 — Cluster Architecture, Installation & Configuration (25%)

25% of the score. This is the domain that separates CKA from CKAD — etcd, kubeadm, RBAC. If you're coming from CKAD, this is all new and it's where I spent the most study time. etcd backup/restore alone took me a week to get reliable. The `--as=system:serviceaccount:ns:name` syntax for testing RBAC was another thing I had to drill until it was automatic.

See also: [Exercise 04 — RBAC](#) / [Exercise 09 — kubeadm Upgrade](#) / [Exercise 18 — CRI-dockerd Setup](#)

4.1 — Manage Role-Based Access Control (RBAC)

RBAC has four objects:

Object	Scope	Binds to
Role	Namespace	RoleBinding
ClusterRole	Cluster-wide	ClusterRoleBinding or RoleBinding
RoleBinding	Namespace	Role or ClusterRole
ClusterRoleBinding	Cluster-wide	ClusterRole

```
# Create Role (namespace-scoped permissions)
k create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  -n dev

# Create RoleBinding
k create rolebinding read-pods \
  --role=pod-reader \
  --serviceaccount=dev:my-sa \
  -n dev

# Create ClusterRole (cluster-wide permissions)
k create clusterrole node-reader \
  --verb=get,list \
  --resource=nodes

# Create ClusterRoleBinding
k create clusterrolebinding read-nodes \
  --clusterrole=node-reader \
  --serviceaccount=dev:my-sa

# Test permissions
k auth can-i list pods -n dev --as=system:serviceaccount:dev:my-sa
k auth can-i list nodes --as=system:serviceaccount:dev:my-sa
```

Tricky bit: a ClusterRole bound with a RoleBinding only grants access in that namespace. A ClusterRole bound with a ClusterRoleBinding grants access cluster-wide. Same ClusterRole, different scope depending on the binding type.

4.2 — Use Kubeadm to Install a Basic Cluster

The kubeadm workflow:

```
# On control plane node:
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Set up kubeconfig
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Install CNI (e.g., Calico)
k apply -f https://docs.projectcalico.org/manifests/calico.yaml

# On worker nodes:
sudo kubeadm join <control-plane-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

If you lost the join command:

```
kubeadm token create --print-join-command
```

4.3 — Manage a Highly Available Kubernetes Cluster

You won't set up HA from scratch on the exam, but they want you to understand the two topologies:

- **Stacked etcd:** etcd on the same nodes as the control plane. This is what everyone uses. Simpler, good enough for most setups.
- **External etcd:** etcd on dedicated nodes. I've never set this up. I doubt you will either. Just know it exists and that it's "more resilient" because etcd failures don't take down the control plane node.
- Multiple control plane nodes behind a load balancer — the LB endpoint goes in `kubeadm init --control-plane-endpoint=<lb>:6443`

I spent maybe 10 minutes on this topic. Read it, understood the difference, moved on. It wasn't worth more time than that.

4.4 — Provision Underlying Infrastructure to Deploy a Kubernetes Cluster

kubeadm handles most of this. You won't provision VMs on the exam, but you might need to fix a node that was set up wrong. The things that break: - containerd not installed or not running — `systemctl status containerd` - Swap still enabled — `swapoff -a` (I always forget this on fresh VMs) - Missing kernel modules: `br_netfilter` and `overlay`. Load them with `modprobe`. - `sysctl` params — `net.bridge.bridge-nf-call-iptables = 1`. I can never remember the exact param name, I just grep the docs page.

4.5 — Perform a Version Upgrade on a Kubernetes Cluster Using Kubeadm

This is a classic CKA question. The sequence matters:

Control plane node:

```
# 1. Update kubeadm
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm

# 2. Plan
sudo kubeadm upgrade plan

# 3. Apply
sudo kubeadm upgrade apply v1.35.0

# 4. Update kubelet + kubect1
sudo apt-mark unhold kubelet kubect1
sudo apt-get install -y kubelet=1.35.0-1.1 kubect1=1.35.0-1.1
sudo apt-mark hold kubelet kubect1

# 5. Restart kubelet
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

Worker node:

```
# 1. From control plane: drain the worker
k drain worker-1 --ignore-daemonsets --delete-emptydir-data

# 2. SSH to worker, update packages
sudo apt-mark unhold kubeadm kubelet kubect1
sudo apt-get update
sudo apt-get install -y kubeadm=1.35.0-1.1 kubelet=1.35.0-1.1 kubect1=1.35.0-1.1
sudo apt-mark hold kubeadm kubelet kubect1

# 3. Upgrade node
sudo kubeadm upgrade node

# 4. Restart kubelet
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# 5. From control plane: uncordon
k uncordon worker-1
```

Key difference: control plane uses `kubeadm upgrade apply`, worker uses `kubeadm upgrade node`.

4.6 — Implement etcd Backup and Restore

This shows up on almost every CKA exam. Memorize this.

Backup:

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

Where to find the cert paths: `cat /etc/kubernetes/manifests/etcd.yaml` and look for `--cert-file`, `--key-file`, `--trusted-ca-file`.

Verify:

```
ETCDCTL_API=3 etcdctl snapshot status /tmp/etcd-backup.db --write-table
```

Restore:

```
# 1. Restore to a new directory
ETCDCTL_API=3 etcdctl snapshot restore /tmp/etcd-backup.db \
--data-dir=/var/lib/etcd-restored

# 2. Update etcd manifest to use the restored directory
sudo vi /etc/kubernetes/manifests/etcd.yaml
# Change --data-dir=/var/lib/etcd → --data-dir=/var/lib/etcd-restored
# Change hostPath path: /var/lib/etcd → /var/lib/etcd-restored

# 3. Wait for etcd to restart (it's a static pod)
# kubect1 may be unresponsive for 30-60 seconds — that's normal
```

The three flags you need every time: `--cacert`, `--cert`, `--key`. I wrote them on the notepad in the exam environment before starting.

Domain 5 — Services & Networking (20%)

20% of the score. Services, Ingress, Gateway API, NetworkPolicy, DNS. NetworkPolicy is where I lost the most points in practice exams — the AND vs OR selector behavior is unintuitive, and forgetting DNS egress is a silent killer.

See also: [Exercise 05 — NetworkPolicy](#) / Skeletons: [service.yaml](#), [ingress.yaml](#), [networkpolicy.yaml](#)

5.1 — Understand Host Networking Configuration on the Cluster Nodes

Networking in Kubernetes "just works" if your CNI is installed correctly. Don't overthink the model — pods get IPs, pods can talk to each other, nodes can talk to pods. The CNI plugin (Calico, Flannel, Cilium) makes it happen. That's really all you need to know conceptually.

What actually matters for the exam: knowing where to look when it doesn't work.

```
# Check what CNI is installed
ls /etc/cni/net.d/
cat /etc/cni/net.d/10-calico.conflist

# Check pod CIDR
k cluster-info dump | grep -m 1 cluster-cidr
```

5.2 — Understand Connectivity Between Pods

Same-node pods talk through a bridge, cross-node pods go through the CNI overlay. You don't need to know the internals — but you need to test connectivity when something breaks.

```
# Test pod-to-pod connectivity
k exec pod-a -- wget -qO- --timeout=2 http://<pod-b-ip>

# Check pod IPs
k get pods -o wide
```

5.3 — Understand ClusterIP, NodePort, LoadBalancer Service Types

Type	How it works	When to use
ClusterIP	Internal cluster IP only	Internal services (default)
NodePort	ClusterIP + port on every node (30000-32767)	Dev/testing, direct node access
LoadBalancer	NodePort + cloud LB	Production with cloud provider
ExternalName	CNAME to external DNS	Pointing to external services

```
# ClusterIP (default)
k expose deployment webapp --port=80 --target-port=8080

# NodePort (random port in 30000-32767)
k expose deployment webapp --port=80 --target-port=8080 --type=NodePort

# Specific NodePort — generate YAML, edit nodePort, then apply
k expose deployment webapp --port=80 --target-port=8080 --type=NodePort $do > svc.yaml
# edit svc.yaml → set spec.ports[0].nodePort: 30080
k apply -f svc.yaml
```

The most important thing: `port` is what clients use to reach the service. `targetPort` is the port the container listens on. `nodePort` is the port on the node (NodePort/LoadBalancer only).

5.4 — Understand How to Use Ingress Controllers and Ingress Resources

Ingress gives you HTTP/HTTPS routing to services based on hostname or path.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-service
            port:
              number: 80

```

Requirements: - An Ingress Controller must be installed (e.g., nginx-ingress). The Ingress resource alone does nothing. - `ingressClassName` is required in v1.35 — the old annotation `kubernetes.io/ingress.class` still works but is deprecated.

5.5 — Know How to Use and Configure CoreDNS

CoreDNS is the cluster DNS server (replaced kube-dns a long time ago). It runs as a Deployment in `kube-system`.

```

# Check CoreDNS pods
k get pods -n kube-system -l k8s-app=kube-dns

# Check CoreDNS config
k get configmap coredns -n kube-system -o yaml

# Test DNS resolution
k run test-dns --image=busybox:1.36 --rm -it -- nslookup kubernetes.default.svc.cluster.local

```

DNS naming: - Service: `<service>.<namespace>.svc.cluster.local` - Pod: `<pod-ip-dashed>.<namespace>.pod.cluster.local`

If DNS doesn't work: 1. Is CoreDNS running? `k get pods -n kube-system -l k8s-app=kube-dns` 2. Does kube-dns service have endpoints? `k get endpoints kube-dns -n kube-system` 3. Is the Corefile correct? `k get cm coredns -n kube-system -o yaml` 4. Can the pod reach the DNS service? `k exec <pod> -- cat /etc/resolv.conf`

5.6 — Choose an Appropriate Container Network Interface Plugin

The CKA won't ask you to write a CNI plugin. Just use Calico. It supports NetworkPolicy, it's the most common, and it's what most training platforms use. The exam doesn't care which CNI is installed.

Other CNIs exist — Flannel is simpler but doesn't support NetworkPolicy (dealbreaker for the exam), Cilium is the fancy eBPF one everyone's talking about, Weave still works. But if a question says "install a CNI plugin," just apply the Calico manifest and move on.

Install is one `kubectl apply -f <url>`. The CNI must be installed before worker nodes join — pods stay Pending without it.

5.7 — Understand NetworkPolicy

NetworkPolicy controls pod-to-pod traffic. Once you apply any NetworkPolicy to a pod, all traffic not explicitly allowed is denied for that pod.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - protocol: TCP
      port: 5432
  # Always allow DNS
  - to: []
    ports:
    - protocol: UDP
      port: 53

```

Critical gotcha on the exam: if you add an Egress policy, you must also allow DNS (UDP 53). Otherwise the pod can't resolve any service names and everything looks broken even though the policy is "correct."

Another gotcha: `from` with multiple selectors in one rule = AND. Multiple rules = OR.

```

# AND – must match BOTH namespace AND pod label
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        env: prod
    podSelector:
      matchLabels:
        app: frontend

# OR – matches namespace OR pod label
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        env: prod
- from:
  - podSelector:
      matchLabels:
        app: frontend

```

This difference tripped me up during practice. Read the indentation carefully.

5.8 – Gateway API (v1.35)

Gateway API is the successor to Ingress. It's GA in v1.35 and may appear on the CKA.

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    allowedRoutes:
      namespaces:
        from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-route
spec:
  parentRefs:
  - name: my-gateway
  hostnames:
  - "myapp.example.com"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api
  backendRefs:
  - name: api-service
    port: 80

```

Key differences from Ingress: - Gateway = infrastructure resource (managed by platform team) - HTTPRoute = application routing (managed by app team) - More features: header matching, traffic splitting, request mirroring - Supports TCP, UDP, gRPC — not just HTTP

CKA Domain Weight Distribution

```

pie title CKA Exam Domain Weights
"Troubleshooting (30%)" : 30
"Cluster Architecture (25%)" : 25
"Services & Networking (20%)" : 20
"Workloads & Scheduling (15%)" : 15
"Storage (10%)" : 10

```

Where to focus: Troubleshooting + Cluster Architecture = 55% of the exam. If you nail these two, you only need a few more points to pass.

Exam Day Strategy — Time Allocation

I used a two-pass approach in practice and it changed everything. Before this, I'd get stuck on a hard question for 12 minutes and run out of time for easy ones worth the same points.

Pass 1 (first 80 minutes): Do all questions in order. If a question looks like it'll take more than 8 minutes, flag it and move on. Don't get emotionally invested in any single question.

Pass 2 (last 40 minutes): Go back to flagged questions. You now know exactly how much time you have. The pressure feels different when you've already banked easy points.

Time estimates by question type:

Question Type	Typical Time	Notes
Create a pod/deployment	2-3 min	Use <code>\$do</code> to generate YAML
Create RBAC resources	3-5 min	Know the imperative commands
NetworkPolicy	5-8 min	Always allow DNS egress
etcd backup/restore	8-10 min	Know the cert paths
kubeadm upgrade	8-10 min	Follow the sequence exactly

Question Type	Typical Time	Notes
Troubleshoot broken node	5-8 min	Check kubelet first
Troubleshoot networking	5-8 min	Check endpoints, selectors
PV/PVC/StorageClass	4-6 min	Match accessModes and storageClassName
DaemonSet	3-4 min	Copy from docs fast
Ingress/Gateway	4-6 min	Know ingressClassName
Static pod	3-4 min	Write manifest directly to /etc/kubernetes/manifests/
Node drain/cordon	2-3 min	--ignore-daemonsets --delete-emptydir-data

Total available: 120 minutes. Budget ~100 minutes for questions, 20 minutes buffer for context switching, copy/paste fumbling, and double-checking.

[Back to top](#)

Mistakes That Will Fail You on the CKA

Every single one of these cost me points during practice exams. I'm not listing hypothetical risks — these are things I actually did wrong and had to learn from.

1. Forgetting to switch context

Every question says "use context k8s-xxx." I missed this twice during one practice exam — answered two questions perfectly on the wrong cluster. Zero points for both. Now I read the context line first, switch, then read the actual question.

```
# ALWAYS do this first
k config use-context <context-name>
```

2. Wrong namespace

I created a perfect Deployment in `default` when the question said `production`. The YAML was right, the containers were right, everything worked — but the grader checks the namespace. Zero points. Now I run `kn <namespace>` as the FIRST command for every question.

```
# Set namespace for the question
kn <namespace>
# Or use -n on every command
k get pods -n production
```

3. YAML indentation errors

I had one tab character hidden in a YAML file. `kubectl apply` gave a cryptic parsing error and I spent 4 minutes hunting for the problem. Set up vim with `expandtab` so tabs become spaces. One wrong indent level = broken YAML = zero points.

```
# This should already be in your .vimrc
set expandtab
set tabstop=2
set shiftwidth=2
```

4. etcd restore — forgetting to update hostPath

This one burned me on my second practice exam. I restored etcd to `/var/lib/etcd-restored` and updated the `--data-dir` flag. Cluster came back but all my previous resources were gone. Turns out the `hostPath.path` in the volume section was still pointing to the old `/var/lib/etcd`. The etcd process and the volume mount have to agree, or you're reading from the wrong directory.

```
# Must update BOTH:
# 1. --data-dir=/var/lib/etcd-restored
# 2. volumes[].hostPath.path: /var/lib/etcd-restored
```

5. Drain without --ignore-daemonsets

`k drain` fails if DaemonSet pods exist and you don't pass the flag. Don't waste time reading the error — just always use:

```
k drain <node> --ignore-daemonsets --delete-emptydir-data
```

6. NetworkPolicy without DNS egress

I have made this mistake more times than I want to admit. You write what looks like a perfect NetworkPolicy egress rule, test connectivity, and it fails. You rewrite the rule. Still fails. You check selectors. Still right. The pod can't resolve service names because you didn't allow UDP 53. I now write the DNS egress block FIRST before writing any other egress rules.

```
# Always include this in egress rules
- to: []
  ports:
  - protocol: UDP
    port: 53
```

7. Not verifying your work

I finished a question early once and moved on feeling confident. Turns out the pod was in `ImagePullBackOff` because I had a typo in the image name. Would have caught it in 5 seconds if I'd checked `k get pod`. Now I verify every single resource I create before moving on.

```
k get pod <name> -n <ns>      # Is it Running?
k get svc <name> -n <ns>      # Does service exist?
k get endpoints <name> -n <ns> # Does service have endpoints?
```

8. Wasting time on hard questions first

A 3-point question and a 7-point question get the same time if you're stuck. Do the easy ones first.

9. Forgetting the ServiceAccount in RBAC

The question says "create a ServiceAccount and give it access." I jumped straight to creating the Role and RoleBinding, then ran `k auth can-i` and it returned "no" for everything. Spent 6 minutes thinking my Role was wrong before realizing the ServiceAccount didn't exist. The RoleBinding referenced a SA that wasn't there. Create the SA first.

```
# Create SA first
k create sa my-sa -n my-ns

# Then bind it
k create rolebinding my-binding \
  --role=my-role \
  --serviceaccount=my-ns:my-sa \
  -n my-ns
```

Vim Keys I Actually Used on Exam Day

Not a vim guide, just the handful I kept hitting. If you already know vim, skip this.

Vim has two modes that matter here: **insert mode** (where you type text like a normal editor) and **normal mode** (where keys are commands). You start in normal mode. Press `i` to enter insert mode, press `Esc` to get back to normal mode. Almost every shortcut below is a normal mode command, so hit `Esc` first if you are not sure where you are.

Opening and saving

- `vim <file>` to open the file. You start in normal mode.
- `i` (normal mode) to enter insert mode and start typing.
- `Esc` to leave insert mode and go back to normal mode.
- `:wq` (normal mode) to save and quit. `:q!` to bail without saving.

Moving around (normal mode)

- `A` jumps to the end of the current line and drops you into insert mode. Handy for adding to an existing line without arrow-keying across.
- `$` moves the cursor to the end of the line without entering insert mode.
- `0` moves to the start of the line.
- `gg` jumps to the top of the file, `G` jumps to the bottom.
- `H` top of the visible screen, `M` middle, `L` bottom.
- `/word` then `Enter` to search, `n` for the next match.

Editing (normal mode)

- `<number>dd` deletes that many lines. `5dd` wipes 5 lines at once. I used this constantly to clean up the junk that `kubectl ... --dry-run=client -o yaml` adds (status blocks, creationTimestamp, empty resources).
- `dd` on its own deletes the current line.
- `u` to undo.

That is all I needed. Anything fancier I would look up, but on exam day I never had to.

[Back to top](#)

Troubleshooting Decision Flowchart

Use this when something is broken and you don't know where to start.

```
flowchart TD
    START[Something is broken] --> NODES{Are all nodes Ready?}

    NODES -->|No| NODE_DEBUG[Node is NotReady]
    NODE_DEBUG --> KUBELET{Is kubelet running?}
    KUBELET -->|No| KUBELET_FIX[systemctl start kubelet<br/>Check journalctl -u kubelet]
    KUBELET -->|Yes| CERTS{Are certs expired?}
    CERTS -->|Yes| CERT_FIX[kubeadm certs renew all]
    CERTS -->|No| NET{Node networking OK?}
    NET -->|No| NET_FIX[Check CNI plugin<br/>Check kube-proxy]

    NODES -->|Yes| PODS{Is the pod running?}

    PODS -->|Pending| PENDING[Check describe pod events]
    PENDING --> PEND_RES{Resource issue?}
    PEND_RES -->|Yes| RES_FIX[Scale down or add nodes]
    PEND_RES -->|No| PEND_SCHED{Scheduling issue?}
    PEND_SCHED -->|Taint| TAINT_FIX[Add toleration or remove taint]
    PEND_SCHED -->|Selector| SEL_FIX[Fix nodeSelector or affinity]
    PEND_SCHED -->|PVC| PVC_FIX[Fix PVC – check storageClass, accessMode]

    PODS -->|CrashLoop| CRASH[Check logs --previous]
    CRASH --> CRASH_CMD{Wrong command?}
    CRASH_CMD -->|Yes| CMD_FIX[Fix command/args]
    CRASH_CMD -->|No| CRASH_CFG{Missing config?}
    CRASH_CFG -->|Yes| CFG_FIX[Fix ConfigMap/Secret/env]
    CRASH_CFG -->|No| APP_FIX[Application bug – check logs]

    PODS -->|ImagePull| IMAGE[Check image name/tag]
    IMAGE --> IMG_SECRET{Private registry?}
    IMG_SECRET -->|Yes| SECRET_FIX[Add imagePullSecrets]
    IMG_SECRET -->|No| IMG_FIX[Fix image name or check network]

    PODS -->|Running| SVC{Can you reach the service?}
    SVC -->|No| EP{Service has endpoints?}
    EP -->|No| LABEL_FIX[Fix selector – labels don't match]
    EP -->|Yes| PORT{targetPort matches container?}
    PORT -->|No| PORT_FIX[Fix targetPort]
    PORT -->|Yes| DNS{DNS resolving?}
    DNS -->|No| DNS_FIX[Check CoreDNS pods + configmap]
    DNS -->|Yes| NETPOL{NetworkPolicy blocking?}
    NETPOL -->|Yes| NP_FIX[Fix NetworkPolicy rules]

    PODS -->|Running| ETCD{etcd healthy?}
    ETCD -->|No| ETCD_FIX[Check etcd pod<br/>Restore from backup if needed]
```

Practice Scenarios with Full Solutions

These are longer, multi-step scenarios that mimic real exam questions.

Scenario 1: etcd Backup and Restore

Task: Back up etcd to `/opt/etcd-backup.db`, then restore it to verify the backup works.

Solution

```
# Find cert paths
grep -E "cert-file|key-file|trusted-ca-file" /etc/kubernetes/manifests/etcd.yaml

# Backup
ETCDCTL_API=3 etcdctl snapshot save /opt/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify
ETCDCTL_API=3 etcdctl snapshot status /opt/etcd-backup.db --write-table

# Restore
ETCDCTL_API=3 etcdctl snapshot restore /opt/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored

# Update etcd manifest
sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-restored|g' /etc/kubernetes/manifests/etcd.yaml

# Wait for etcd to restart
sleep 30
k get nodes
```

Scenario 2: RBAC — ServiceAccount with Limited Access

Task: In namespace `dev`, create a ServiceAccount `deploy-bot` that can only create and list Deployments. Verify it cannot delete pods.

Solution

```
k create ns dev
k create sa deploy-bot -n dev
k create role deploy-manager -n dev \
  --verb=create,list,get \
  --resource=deployments
k create rolebinding deploy-bot-binding -n dev \
  --role=deploy-manager \
  --serviceaccount=dev:deploy-bot

# Verify
k auth can-i create deployments -n dev --as=system:serviceaccount:dev:deploy-bot
# yes
k auth can-i delete pods -n dev --as=system:serviceaccount:dev:deploy-bot
# no
```

Scenario 3: Node Drain and Maintenance

Task: Drain node `worker-2` for maintenance, verify pods are rescheduled, then bring it back.

Solution

```
# Check current state
k get pods -o wide | grep worker-2

# Drain
k drain worker-2 --ignore-daemonsets --delete-emptydir-data

# Verify node is cordoned
k get nodes
# worker-2 should show SchedulingDisabled

# Verify pods moved
k get pods -o wide
# No non-DaemonSet pods on worker-2

# Simulate maintenance (wait)
# ...

# Bring back
k uncordon worker-2
k get nodes
# worker-2 should be Ready
```

Scenario 4: kubeadm Upgrade

Task: Upgrade the control plane from v1.34.x to v1.35.0.

Solution

```
# 1. Upgrade kubeadm
sudo apt-mark unhold kubeadm
sudo apt-get update
sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm

# 2. Plan
sudo kubeadm upgrade plan

# 3. Apply
sudo kubeadm upgrade apply v1.35.0

# 4. Upgrade kubelet + kubect1
sudo apt-mark unhold kubelet kubect1
sudo apt-get install -y kubelet=1.35.0-1.1 kubect1=1.35.0-1.1
sudo apt-mark hold kubelet kubect1

# 5. Restart
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# 6. Verify
k get nodes
```

Scenario 5: Troubleshoot — Pod Can't Reach Service

Task: Pod `client` can't reach service `backend-svc` on port 80. Find and fix the issue.

Solution

```
# 1. Check service exists
k get svc backend-svc

# 2. Check endpoints
k get endpoints backend-svc
# If empty: selector doesn't match!

# 3. Check service selector
k describe svc backend-svc | grep Selector
# e.g., Selector: app=backend

# 4. Check pod labels
k get pods --show-labels | grep backend
# e.g., labels are app=back (typo!)

# 5. Fix pod labels
k label pod <backend-pod> app=backend --overwrite

# 6. Or fix service selector
k edit svc backend-svc
# Change selector to match actual pod labels

# 7. Verify
k get endpoints backend-svc
# Should now show pod IPs
k exec client -- wget -qO- --timeout=2 http://backend-svc
```

Scenario 6: NetworkPolicy — Isolate Database

Task: Create a NetworkPolicy that allows only pods with label `role=api` to reach pods with label `role=db` on port 5432. Block all other ingress to the database.

Solution

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-isolation
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: api
      ports:
        - protocol: TCP
          port: 5432
```

```
k apply -f db-policy.yaml

# Test - api pod should work
k exec api-pod -- nc -zv <db-pod-ip> 5432

# Test - other pod should fail
k exec other-pod -- nc -zv <db-pod-ip> 5432
```

Scenario 7: PV/PVC – Persistent Storage

Task: Create a PV using hostPath `/data/logs` (1Gi, ReadWriteOnce, Retain), a PVC requesting 500Mi, and mount it in a pod at `/var/log/app`.

Solution

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: log-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/logs
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: log-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
  storageClassName: manual
---
apiVersion: v1
kind: Pod
metadata:
  name: log-pod
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do echo $(date) >> /var/log/app/app.log; sleep 5; done"]
      volumeMounts:
        - name: log-vol
          mountPath: /var/log/app
  volumes:
    - name: log-vol
      persistentVolumeClaim:
        claimName: log-pvc

```

```

k apply -f storage.yaml
k get pv log-pv
k get pvc log-pvc
k exec log-pod -- cat /var/log/app/app.log

```

Scenario 8: Sidecar Container — Log Streaming

Task: Create a pod with a main container writing logs to `/var/log/app/app.log` and a sidecar (native v1.35 style) streaming that file to stdout.

Solution

```

apiVersion: v1
kind: Pod
metadata:
  name: sidecar-logging
spec:
  initContainers:
    - name: log-streamer
      image: busybox:1.36
      restartPolicy: Always
      command: ["sh", "-c", "tail -f /var/log/app.log"]
      volumeMounts:
        - name: log-vol
          mountPath: /var/log
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do echo \"$(date) app running\" >> /var/log/app.log; sleep 3; done"]
      volumeMounts:
        - name: log-vol
          mountPath: /var/log
  volumes:
    - name: log-vol
      emptyDir: {}

```

```

k apply -f sidecar.yaml
k logs sidecar-logging -c log-streamer

```

Practice Questions with Answers (Mock Exam)

17 questions weighted to match the real exam. Switch context before each one.

Question 1 [4%] [Cluster Architecture] Easy

```
kubectl config use-context k8s-cluster1
```

Create a ServiceAccount named `monitoring-sa` in namespace `monitoring`. Create a ClusterRole named `pod-viewer` that can `get`, `list`, `watch` pods in all namespaces. Bind the ClusterRole to the ServiceAccount.

Solution

```

k create ns monitoring
k create sa monitoring-sa -n monitoring
k create clusterrole pod-viewer --verb=get,list,watch --resource=pods
k create clusterrolebinding pod-viewer-binding \
  --clusterrole=pod-viewer \
  --serviceaccount=monitoring:monitoring-sa

# Verify
k auth can-i list pods -A --as=system:serviceaccount:monitoring:monitoring-sa

```

Question 2 [5%] [Cluster Architecture] Medium

```
kubectl config use-context k8s-cluster1
```

Back up etcd to `/opt/etcd-snapshot.db`. The etcd server is running on the control plane node at `https://127.0.0.1:2379`.

Solution

```

# Find cert paths
cat /etc/kubernetes/manifests/etcd.yaml | grep -E "cert-file|key-file|trusted-ca"

ETCDCTL_API=3 etcdctl snapshot save /opt/etcd-snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify
ETCDCTL_API=3 etcdctl snapshot status /opt/etcd-snapshot.db --write-table

```

Question 3 [3%] [Workloads & Scheduling] Easy

```
kubectl config use-context k8s-cluster2
```

Create a Deployment named `web-app` in namespace `production` with 3 replicas using image `nginx:1.27`. Expose it as a ClusterIP service named `web-svc` on port 80.

Solution

```
k create ns production
k create deployment web-app -n production --image=nginx:1.27 --replicas=3
k expose deployment web-app -n production --port=80 --target-port=80 --name=web-svc
k get deploy,svc -n production
```

Question 4 [5%] [Troubleshooting] Medium

```
kubectl config use-context k8s-cluster1
```

Node `worker-1` is showing `NotReady`. Investigate and fix the issue so the node becomes `Ready`.

Solution

```
# Check node status
k describe node worker-1 | grep -A5 Conditions

# SSH to the node
ssh worker-1

# Check kubelet
sudo systemctl status kubelet
# If inactive/dead:
sudo systemctl start kubelet
sudo systemctl enable kubelet

# If it's a config issue, check logs
sudo journalctl -u kubelet --no-pager | tail -30

# Common fixes:
# - Wrong --kubeconfig path
# - Certificate issue → check /var/lib/kubelet/config.yaml
# - Swap enabled → sudo swapoff -a

# Verify from control plane
k get nodes
```

Question 5 [4%] [Services & Networking] Medium

```
kubectl config use-context k8s-cluster2
```

Create a NetworkPolicy named `restrict-ingress` in namespace `production` that: - Applies to pods with label `app=database` - Allows ingress only from pods with label `app=backend` on TCP port 3306 - Allows DNS egress (UDP 53)

Solution

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-ingress
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 3306
  egress:
    - to: []
      ports:
        - protocol: UDP
          port: 53

```

```
k apply -f netpol.yaml
```

Question 6 [5%] [Cluster Architecture] Medium

```
kubectl config use-context k8s-cluster1
```

Upgrade the control plane node from Kubernetes v1.34.4 to v1.35.0 using kubeadm.

Solution

```

sudo apt-mark unhold kubeadm
sudo apt-get update
sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm

sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.35.0

sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.35.0-1.1 kubectl=1.35.0-1.1
sudo apt-mark hold kubelet kubectl

sudo systemctl daemon-reload
sudo systemctl restart kubelet

k get nodes

```

Question 7 [3%] [Storage] Easy

```
kubectl config use-context k8s-cluster2
```

Create a PersistentVolume named `data-pv` with 2Gi capacity, ReadWriteOnce access, hostPath `/data/volumes/pv1`, and storageClassName `manual`. Create a PersistentVolumeClaim named `data-pvc` in namespace `storage-test` requesting 1Gi.

Solution

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: data-pv
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  hostPath:
    path: /data/volumes/pv1
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
  namespace: storage-test
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual

```

```

k create ns storage-test
k apply -f storage.yaml
k get pv data-pv
k get pvc data-pvc -n storage-test

```

Question 8 [5%] [Troubleshooting] Hard

`kubectl config use-context k8s-cluster1`

Service `frontend-svc` in namespace `web` has no endpoints. Pods with label `app=frontend` are running. Find and fix the issue.

Solution

```

# Check service
k describe svc frontend-svc -n web | grep Selector
# e.g., Selector: app=front (typo)

# Check pod labels
k get pods -n web --show-labels
# Labels show: app=frontend

# Fix: edit service selector
k edit svc frontend-svc -n web
# Change selector from app=front to app=frontend
# Or:
k patch svc frontend-svc -n web -p '{"spec":{"selector":{"app":"frontend"}}}'

# Verify
k get endpoints frontend-svc -n web

```

Question 9 [4%] [Workloads & Scheduling] Medium

`kubectl config use-context k8s-cluster2`

Create a DaemonSet named `log-collector` in namespace `kube-system` using image `fluentd:v1.17`. It should run on all nodes including the control plane.

Solution

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app: log-collector
  template:
    metadata:
      labels:
        app: log-collector
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: fluentd
          image: fluentd:v1.17

```

```

k apply -f ds.yaml
k get ds log-collector -n kube-system
k get pods -n kube-system -l app=log-collector -o wide
# Should run on ALL nodes

```

Question 10 [5%] [Troubleshooting] Hard

```
kubectl config use-context k8s-cluster1
```

DNS resolution is not working in the cluster. Pods cannot resolve service names. Find and fix the issue.

Solution

```

# 1. Check CoreDNS pods
k get pods -n kube-system -l k8s-app=kube-dns
# Are they running? CrashLoopBackOff?

# 2. If not running, check logs
k logs -n kube-system -l k8s-app=kube-dns

# 3. Check CoreDNS ConfigMap
k get cm coredns -n kube-system -o yaml
# Look for syntax errors in Corefile

# 4. Check kube-dns service
k get svc kube-dns -n kube-system
k get endpoints kube-dns -n kube-system

# 5. Common fixes:
# - Corefile syntax error → fix ConfigMap, pods restart automatically
# - CoreDNS pods not scheduled → check tolerations
# - kube-dns service missing → recreate it

# 6. Test
k run test-dns --image=busybox:1.36 --rm -it -- nslookup kubernetes.default.svc.cluster.local

```

Question 11 [3%] [Workloads & Scheduling] Easy

```
kubectl config use-context k8s-cluster2
```

Create a static pod named `static-nginx` on node `worker-1` using image `nginx:1.27` with port 80.

Solution

```
# SSH to worker-1
ssh worker-1

# Create manifest
sudo tee /etc/kubernetes/manifests/static-nginx.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: static-nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
EOF

# Back on control plane
k get pods | grep static-nginx
```

Question 12 [4%] [Cluster Architecture] Medium

```
kubectl config use-context k8s-cluster1
```

Drain node `worker-2` for maintenance. Make sure no pods are disrupted unexpectedly. After a simulated maintenance window, make the node schedulable again.

Solution

```
k drain worker-2 --ignore-daemonsets --delete-emptydir-data

# Verify
k get nodes
# worker-2: SchedulingDisabled
k get pods -o wide | grep worker-2
# Only DaemonSet pods

# After maintenance
k uncordon worker-2
k get nodes
# worker-2: Ready
```

Question 13 [5%] [Services & Networking] Medium

```
kubectl config use-context k8s-cluster2
```

Create an Ingress resource named `app-ingress` in namespace `web` that routes: - `app.example.com/api` to service `api-svc` on port 8080 - `app.example.com/web` to service `web-svc` on port 80

Use `ingressClassName` `nginx`.

Solution

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: web
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 8080
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-svc
            port:
              number: 80

```

```

k apply -f ingress.yaml
k get ingress app-ingress -n web

```

Question 14 [5%] [Troubleshooting] Hard

```
kubectl config use-context k8s-cluster1
```

A pod named `failing-app` in namespace `debug` is in `CrashLoopBackOff`. Investigate and fix it.

Solution

```

# 1. Check events
k describe pod failing-app -n debug

# 2. Check logs (previous container)
k logs failing-app -n debug --previous

# 3. Common issues:
# a) Wrong command → fix command/args
# b) Missing ConfigMap/Secret → create the missing resource
# c) Wrong env var reference → fix valueFrom
# d) Application crash → fix the image or config

# 4. Fix based on what you find, e.g.:
k edit pod failing-app -n debug
# Or delete and recreate with fixes:
k get pod failing-app -n debug -o yaml > fix.yaml
# Edit fix.yaml
k delete pod failing-app -n debug $now
k apply -f fix.yaml

# 5. Verify
k get pod failing-app -n debug
# Should be Running

```

Question 15 [4%] [Storage] Medium

```
kubectl config use-context k8s-cluster2
```

Create a pod named `volume-pod` in namespace `storage-test` that mounts the existing PVC `data-pvc` at `/data`. Write the string "exam-test" to `/data/test.txt`. Verify the file exists.

Solution

```

apiVersion: v1
kind: Pod
metadata:
  name: volume-pod
  namespace: storage-test
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "echo 'exam-test' > /data/test.txt && sleep 3600"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-pvc

```

```

k apply -f pod.yaml
k exec volume-pod -n storage-test -- cat /data/test.txt
# Should output: exam-test

```

Question 16 [5%] [Cluster Architecture] Hard

```
kubectl config use-context k8s-cluster1
```

Restore etcd from the snapshot at `/opt/etcd-snapshot.db`. Restore it to data directory `/var/lib/etcd-from-backup`.

Solution

```

# Restore
ETCDCTL_API=3 etcdctl snapshot restore /opt/etcd-snapshot.db \
  --data-dir=/var/lib/etcd-from-backup

# Update etcd manifest
sudo vi /etc/kubernetes/manifests/etcd.yaml
# 1. Change --data-dir=/var/lib/etcd to --data-dir=/var/lib/etcd-from-backup
# 2. In volumes section, change hostPath path from /var/lib/etcd to /var/lib/etcd-from-backup

# Wait for etcd to restart
# kubectl may hang for 30-60s - that's normal
sleep 30
k get nodes

```

Question 17 [4%] [Workloads & Scheduling] Medium

```
kubectl config use-context k8s-cluster2
```

Create a pod named `restricted-pod` in namespace `secure` that: - Uses image `nginx:1.27` - Runs as user ID 1000 - Has a read-only root filesystem - Drops all capabilities - Does not allow privilege escalation

Solution

```

apiVersion: v1
kind: Pod
metadata:
  name: restricted-pod
  namespace: secure
spec:
  securityContext:
    runAsUser: 1000
  containers:
  - name: nginx
    image: nginx:1.27
    securityContext:
      readOnlyRootFilesystem: true
      allowPrivilegeEscalation: false
      capabilities:
        drop:
        - ALL

```

```

k create ns secure
k apply -f restricted.yaml
k get pod restricted-pod -n secure

```

Score Card

Copy this table into a text file. After completing the mock, mark each question as full credit, partial, or missed. Add up your weighted score.

#	Domain	Weight	Difficulty	Result	Points
1	Cluster Architecture	4%	Easy	___	/4
2	Cluster Architecture	5%	Medium	___	/5
3	Workloads & Scheduling	3%	Easy	___	/3
4	Troubleshooting	5%	Medium	___	/5
5	Services & Networking	4%	Medium	___	/4
6	Cluster Architecture	5%	Medium	___	/5
7	Storage	3%	Easy	___	/3
8	Troubleshooting	5%	Hard	___	/5
9	Workloads & Scheduling	4%	Medium	___	/4
10	Troubleshooting	5%	Hard	___	/5
11	Workloads & Scheduling	3%	Easy	___	/3
12	Cluster Architecture	4%	Medium	___	/4
13	Services & Networking	5%	Medium	___	/5
14	Troubleshooting	5%	Hard	___	/5
15	Storage	4%	Medium	___	/4
16	Cluster Architecture	5%	Hard	___	/5
17	Workloads & Scheduling	4%	Medium	___	/4
	Total	73%			___/73

How to score: Full credit = full weight. Partial (got the right idea but missed a flag or namespace) = half weight. Wrong or skipped = 0. Real exam uses partial scoring too, so this is realistic.

Passing threshold: 66% of 73 = **48 points**. If you're below 48, re-study the domains where you dropped points and redo those questions in a week.

Timing target: Set a timer for 2 hours. If you finish early, note how much time you had left — that buffer is your safety margin on exam day.

Domain breakdown of this mock:

Domain	Questions	Total Weight	Your Score
Cluster Architecture	1, 2, 6, 12, 16	23%	___/23
Troubleshooting	4, 8, 10, 14	20%	___/20
Services & Networking	5, 13	9%	___/9
Workloads & Scheduling	3, 9, 11, 17	14%	___/14
Storage	7, 15	7%	___/7

If any domain is below 50%, that's where your next study session should focus.

[Back to top](#)

Study Resources for CKA 2026

What I actually used, in order of usefulness:

Free

Resource	What I actually used it for
Kubernetes Official Docs	The only site allowed during the exam. I spent a week getting fast at searching it. Bookmark the Tasks section — that's where etcd backup and kubeadm upgrade live.
Kubernetes Tasks	This saved me on the etcd question during the actual exam. I had the steps bookmarked and copied the cert paths directly. Don't skip this.
kubect! Cheat Sheet	I had this open in a tab during the exam. The jsonpath examples alone are worth bookmarking.
CKA Curriculum PDF	Checked this the night before to make sure I hadn't missed a topic. Found out I'd skipped Pod Scheduling Readiness entirely.
Killercoda CKA Scenarios	Free browser labs. I did 2-3 of these per day during lunch. The RBAC and NetworkPolicy ones were the most useful.

Paid

Resource	Honest review	Cost
killer.sh	Included with your exam purchase. Two 24-hour sessions. Significantly harder, but the real exam also has tricky questions. If you can pass killer.sh, you're likely ready. I scored 60% on killer.sh and 89% on the actual CKA. Don't waste both sessions early. Save one for the week before.	Free with exam
KodeKloud CKA Course	Mumshad's course carried me through the first two weeks. The built-in labs are what make it worth it — I wouldn't have learned etcd restore without them.	~\$15-25/mo
Udemy — Mumshad CKA	Same content as KodeKloud but one-time purchase. Wait for a Udemy sale (\$10-15). I used KodeKloud instead, but either works.	~\$15

Local Practice Environments

Tool	My experience
kind	This was my daily driver. Create/destroy clusters in seconds. Multi-node clusters for testing drain/cordon. Only downside: no systemd inside containers, so you can't practice kubelet restart.
minikube	Single-node, easy setup. Fine for basic exercises but useless for kubeadm upgrade or node troubleshooting.
kubeadm	The real deal. Set up actual VMs (I used Vagrant + VirtualBox) and install with kubeadm. Painful to set up, but this is how you actually learn the cluster architecture domain.

Advice: don't spend your first 2 weeks watching videos. Spend 30% of your time on theory and 70% hands-on in a real cluster.

CKA Study Plan (4-5 Weeks)

This is roughly what I followed. I deviated a lot — Week 3 was supposed to be networking but I was still struggling with etcd restore, so I spent half that week redoing exercise 07 until I could do it without the docs. Adjust as you go. If something isn't clicking, stay on it.

Week 1 — Foundations

- Set up a practice cluster (kind or minikube)
- Configure aliases and vim ([exam-setup.sh](#))
- Cover Domain 3 (Workloads & Scheduling): Pods, Deployments, ConfigMaps, Secrets, scheduling
- Do exercises 01-03, 06
- Practice generating YAML with `$do` — you should never type a full manifest from scratch

Week 2 — Cluster Administration

- Cover Domain 4 (Cluster Architecture): RBAC, kubeadm, etcd backup/restore
- Set up a multi-node cluster with kubeadm (VMs or cloud)
- Do exercises 04, 07, 08, 09, 10
- Practice etcd backup/restore until you can do it from memory
- Memorize RBAC imperative commands (create role, create rolebinding, auth can-i)

Week 3 — Networking + Storage + Troubleshooting

- Cover Domain 5 (Services & Networking): Services, Ingress, NetworkPolicy, CoreDNS
- Cover Domain 1 (Storage): PV, PVC, StorageClass
- Cover Domain 2 (Troubleshooting): start breaking things and fixing them
- Do exercises 05, 11, 12
- Do killer.sh session 1
- This was my hardest week. NetworkPolicy AND storage AND troubleshooting is a lot. If you need to push something to Week 4, push storage.

Week 4 — Mock Exams + Weak Areas

- Do the [mock exam](#) in this repo under timed conditions (2 hours)
- Review killer.sh results — identify weak areas
- Redo any exercises you struggled with
- Practice context switching discipline — every question, switch context first
- Do killer.sh session 2 (3 days before exam)

Week 5 (Optional) — Polish

- Only if you didn't feel ready after Week 4
- Focus entirely on weak domains
- Practice speed: can you do a full RBAC setup in under 3 minutes?
- Review the [mistakes list](#) one more time
- I didn't need this week, but I scheduled it as a buffer just in case

killer.sh vs the Real CKA Exam

I did killer.sh twice. Here's how it compares:

	killer.sh	Real CKA Exam
Difficulty	Harder — deliberately over-tests	Moderate
Number of questions	~25	~17-25
Time pressure	Very tight — most people don't finish	Tight but doable
Question length	Some are multi-step and long	More focused, but some still tricky
Scoring	Shows score after 24h session	Shows score in 24h via email
Environment	Same PSI-like terminal	PSI Secure Browser
kubectl access	Same as real exam	Same
Docs access	kubernetes.io	kubernetes.io
Gotchas	Design to fail you	Fewer gotchas, but present

My scores: - killer.sh session 1: 62% (failed, felt terrible) - killer.sh session 2: 78% (passed, felt confident) - Real exam: 89%

If you score 60%+ on killer.sh, you'll likely pass the real exam. Note: killer.sh is harder overall, but the real exam has its own tricky questions. Don't underestimate the actual exam — stay focused on every question, not just the complex ones.

[Back to top](#)

CKA Exam Day Checklist

1 Week Before

- [] Schedule the exam — pick a time when you're sharpest, not Friday evening after a long week (I did Saturday 10am)
- [] Verify your CNCF account name matches your government ID EXACTLY — character for character, including middle name if it's on your ID
- [] Do killer.sh session 2
- [] Review your weakest domain one more time — for me that was NetworkPolicy and etcd
- [] Test your webcam, microphone, and internet — the PSI system check catches issues early

1 Day Before

- [] Clear your desk completely — nothing on it except laptop, keyboard, mouse. I had to remove a sticky note from my monitor.
- [] Remove second monitors, disconnect external screens

- ☐ Switch to wired ethernet if possible — WiFi dropped during my `killer.sh` practice and I lost 2 minutes
- ☐ Run through the [first 60 seconds setup](#) from memory one last time
- ☐ Read the [exam day strategy](#) but don't cram new content — it won't stick
- ☐ Sleep. Seriously. I went to bed early and it helped more than any last-minute studying.

30 Minutes Before

- ☐ Close every app except the PSI browser — it checks for background processes
- ☐ Go to the bathroom. You can't pause the exam.
- ☐ Have water ready in a clear bottle with no label (they make you peel it off if it has one)
- ☐ Start PSI check-in 15 minutes early — the room scan and ID verification took longer than I expected
- ☐ Show the proctor your desk, under your desk, and your walls

During the Exam

- ☐ Type your aliases and vim config FIRST — before even reading question 1
- ☐ Switch context before every question — I lost two answers to wrong context during practice
- ☐ Read the full question before touching the terminal. I misread a question once and solved the wrong problem.
- ☐ Flag anything that looks like it'll take >8 minutes. Come back after you've scored the easy points.
- ☐ Verify everything: `k get`, `k describe`, check the namespace
- ☐ If `kubectl` hangs after `etcd` restore, WAIT. It comes back in 30-60 seconds. Don't start editing the manifest again.

Study Progress Tracker

Track your progress across all CKA domains.

Domain 1 — Storage (10%)

- ☐ Understand `PersistentVolume` and `PersistentVolumeClaim`
- ☐ Understand `StorageClass` and dynamic provisioning
- ☐ Know access modes (RWO, ROX, RWX, RWOP)
- ☐ Know reclaim policies (Retain, Delete)
- ☐ CSI driver basics and troubleshooting
- ☐ Mount PVC in a Pod
- ☐ Complete [Exercise 12](#)

Domain 2 — Troubleshooting (30%)

- ☐ Read `kubelet` logs with `journalctl`
- ☐ Check control plane pod logs
- ☐ Debug Pending pods
- ☐ Debug `CrashLoopBackOff` pods
- ☐ Debug `ImagePullBackOff`
- ☐ Troubleshoot Service endpoints
- ☐ Troubleshoot CoreDNS
- ☐ Troubleshoot NetworkPolicy
- ☐ Use `kubectl debug` (ephemeral containers + node debug)
- ☐ Complete [Exercise 11](#), [17](#)

Domain 3 — Workloads & Scheduling (15%)

- ☐ Create and manage Deployments
- ☐ Rolling update and rollback
- ☐ `ConfigMaps` and `Secrets` (env and volume)
- ☐ Resource requests and limits
- ☐ HPA (autoscaling/v2)
- ☐ `DaemonSet`
- ☐ Static pods
- ☐ `nodeSelector` and node affinity
- ☐ Taints and tolerations
- ☐ Complete [Exercise 01](#), [06](#), [10](#), [16](#)

Domain 4 — Cluster Architecture (25%)

- ☐ RBAC: `Role`, `ClusterRole`, `RoleBinding`, `ClusterRoleBinding`
- ☐ `kubectl auth can-i` for RBAC debugging
- ☐ `ServiceAccounts`

- ☐ Pod Security Standards (PSS) enforcement
- ☐ kubeadm cluster setup
- ☐ kubeadm cluster upgrade
- ☐ Container runtime configuration (CRI-dockerd, containerd)
- ☐ Understand HA topology
- ☐ Helm install, upgrade, rollback
- ☐ Kustomize base + overlay
- ☐ Complete [Exercise 04](#), [08](#), [09](#), [13](#), [14](#), [18](#), [20](#)

Domain 5 — Services & Networking (20%)

- ☐ ClusterIP, NodePort, LoadBalancer services
- ☐ Classic Ingress resources and path-based routing
- ☐ Ingress TLS termination and IngressClass
- ☐ Gateway API (Gateway + HTTPRoute)
- ☐ NetworkPolicy (ingress + egress)
- ☐ CoreDNS configuration
- ☐ CNI plugin awareness
- ☐ Complete [Exercise 05](#), [15](#), [19](#)

Exam Readiness

- ☐ Aliases and vim config memorized
- ☐ CRI-dockerd kernel configuration and setup memorized
- ☐ Can install and configure kubeadm cluster from memory
- ☐ YAML skeletons written from memory (at least 10/23)
- ☐ killer.sh session 1 completed
- ☐ killer.sh session 2 completed
- ☐ Mock exam completed (>66%)
- ☐ ID verified and CNCF account name matches

Mock Exams — Final Preparation

Two comprehensive practice exams matching real CKA format:

- [Mock Exam 01](#) — 15 questions, 2 hours (answers in [MOCK-EXAM-01-SOLUTIONS.md](#))
- [Mock Exam 02](#) — 15 questions, 2 hours (answers in [MOCK-EXAM-02-SOLUTIONS.md](#))

Each exam: - Covers all 5 domains with realistic weight distribution - Requires 7-10 minutes per question (like real exam) - Has separate question and solution files (don't look at solutions until done) - Focuses on integration across domains, not single-domain skills

Scoring: 10+ correct (66%) = pass. 12+ = strong. 15/15 = ready for the real exam.

Study approach: Complete all 22 exercises first, then take both mock exams under timed conditions. Track weak areas and review corresponding exercises before taking the real exam.

YAML Skeletons — Write These from Memory

These are in the [skeletons/](#) directory. During the exam, I wrote most of these from memory instead of copying from docs — it was faster.

Pod

```

apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-pod
spec:
  containers:
  - name: main
    image: nginx:1.27
    ports:
    - containerPort: 80
  resources:
    requests:
      memory: "64Mi"
      cpu: "250m"
    limits:
      memory: "128Mi"
      cpu: "500m"

```

Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-deployment
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: my-deployment
    spec:
      containers:
      - name: main
        image: nginx:1.27
        ports:
        - containerPort: 80

```

Service (ClusterIP + NodePort)

```

apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-deployment
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-nodeport
spec:
  type: NodePort
  selector:
    app: my-deployment
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080

```

NetworkPolicy

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: my-netpol
spec:
  podSelector:
    matchLabels:
      app: my-app
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 80
  egress:
    - to:
        - podSelector:
            matchLabels:
              role: db
      ports:
        - protocol: TCP
          port: 5432
    - to: []
      ports:
        - protocol: UDP
          port: 53

```

PV + PVC

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/my-pv
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual

```

Ingress

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-service
            port:
              number: 80

```

RBAC (Role + RoleBinding)

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-sa
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

ClusterRole + ClusterRoleBinding

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "watch", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-nodes
subjects:
- kind: ServiceAccount
  name: monitoring-sa
  namespace: monitoring
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io

```

DaemonSet

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: my-daemonset
spec:
  selector:
    matchLabels:
      app: my-daemonset
  template:
    metadata:
      labels:
        app: my-daemonset
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: agent
          image: fluentd:v1.17

```

StatefulSet

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-statefulset
spec:
  serviceName: my-headless
  replicas: 3
  selector:
    matchLabels:
      app: my-statefulset
  template:
    metadata:
      labels:
        app: my-statefulset
    spec:
      containers:
        - name: main
          image: nginx:1.27
          volumeMounts:
            - name: data
              mountPath: /data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi

```

Job + CronJob

```

apiVersion: batch/v1
kind: Job
metadata:
  name: my-job
spec:
  completions: 3
  parallelism: 2
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: worker
        image: busybox:1.36
        command: ["sh", "-c", "echo done"]
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: my-cronjob
spec:
  schedule: "*/5 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: cron
            image: busybox:1.36
            command: ["sh", "-c", "date"]

```

ConfigMap + Secret

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_MODE: "production"
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
stringData:
  DB_USER: admin
  DB_PASS: changeme

```

SecurityContext

```

apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  containers:
  - name: main
    image: busybox:1.36
    command: ["sleep", "3600"]
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL

```

ResourceQuota + LimitRange

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: my-quota
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: my-limits
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "64Mi"

```

StorageClass

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: kubernetes.io/no-provisioner
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer

```

Gateway API (Gateway + HTTPRoute)

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    allowedRoutes:
      namespaces:
        from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-route
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: my-service
      port: 80

```

Is the CKA exam hard?

Yes. Harder than I expected. The questions themselves aren't insane, but doing 17 tasks in 2 hours on a laggy remote desktop is stressful. I ran out of time on my first `killer.sh` attempt and only finished 12 questions. The passing score is 66%, which sounds low until you realize you're typing YAML from memory while a timer counts down. Practice until you're fast, not just correct.

Is the CKA worth it in 2026?

For me, absolutely. Not because of the badge — because studying for it forced me to learn `etcd`, `kubeadm`, and cluster troubleshooting. I'd been deploying apps to Kubernetes for a year without understanding any of that. The cert also gets you past resume filters for platform/SRE roles. Every job posting I looked at listed CKA. Whether you need the cert or just the knowledge depends on your situation, but I'd do it again.

How long should I study for the CKA?

It took me about 4 weeks. I was already deploying apps to Kubernetes at work but had never touched `etcd` or `kubeadm`, so those domains ate most of my study time. If you're already an admin, less. If you're brand new to Kubernetes, honestly double it — you need to learn Kubernetes itself before you learn the exam material. I wouldn't try to rush it under 3 weeks unless you're very comfortable already.

Do I need any prerequisites for the CKA?

Officially no. Unofficially, if you can't `vim` a file, `ssh` into a server, or read YAML without your eyes glazing over, you're going to have a bad time. The exam assumes you already know: - Linux command line — `bash`, `vim`, `systemctl`, `journalctl`. You will live in the terminal. - Basic networking — DNS, ports, TCP. Not deep stuff, but you need to understand why pod-to-service traffic uses port 53. - YAML — one wrong indent and nothing works. Practice until you can spot indentation errors by looking.

You don't need Docker experience specifically. `containerd` is the runtime now. Docker knowledge helps but isn't required.

What if I fail the CKA?

You get one free retake with the \$445 purchase. Use it. Seriously — a lot of people fail the first attempt, especially on time management. I know people who scored 55% the first time and 80%+ on the retake just because they knew what to expect. The retake window is 12 months, so there's no rush.

Is the CKA exam open book?

Kind of. You can open kubernetes.io/docs, kubernetes.io/blog, and github.com/kubernetes during the exam. That's it. No Stack Overflow, no personal notes, no ChatGPT. I bookmarked the `etcd` backup page and the `kubeadm` upgrade page before the exam — saved me at least 3 minutes of searching. The docs are allowed but the search is slow, so know where things are before exam day.

Can I take the CKA remotely?

Yes, it's remote-only via PSI Secure Browser. You need a quiet room, clear desk, webcam, and mic. The proctor watches you the entire time. I had to show my entire desk and the area under it before starting. One tip: use a wired internet connection. My WiFi dropped during a `killer.sh` practice run and I lost 2 minutes reconnecting. I switched to ethernet for the real thing.

Should I take CKA or CKAD first?

CKA if you manage clusters or want to. CKAD if you only deploy apps and never touch the infrastructure. I did CKA first because I wanted to understand the whole stack, not just the deployment side. About 40% of the content overlaps (pods, services, deployments), so whichever you do second is noticeably easier.

How is the CKA different from CKS?

CKA is about building and fixing clusters. CKS is about locking them down — Falco rules, AppArmor profiles, OPA policies, audit logging. CKS requires an active CKA to even register. Unless your job is specifically Kubernetes security, I'd skip CKS and focus on getting real cluster experience instead.

How many questions are on the CKA?

Somewhere around 17-25 tasks. Mine had 17. Each task has a weight percentage — a 7% question is worth more than a 3% one, obviously. I did the high-weight questions first on my second pass. Don't treat all questions equally.

What Kubernetes version is on the CKA in 2026?

v1.35 as of March 2026. They update it to match recent stable releases. Check the [CNCF handbook](#) before your exam — if you studied on v1.34, most things are the same but native sidecars and Gateway API are GA now, and those showed up on my exam.

Can I use aliases and scripts during the CKA?

Yes. You can set up any aliases, bash functions, or vim config you want at the start of the exam. The exam environment gives you a fresh terminal — set it up before starting questions.

How fast do I get my CKA results?

Results come via email within 24 hours. Mine arrived in about 12 hours. You'll get a score and pass/fail. If you pass, the certificate PDF is available in your CNCF portal.

What's the pass rate for the CKA?

No idea. The CNCF doesn't publish it. Most people I talked to in r/kubernetes and the CNCF Slack passed on the first or second try. If you've done killer.sh and scored 60%+, you'll be fine.

Final Words

The stuff I practiced was the stuff that showed up. If something in this guide is wrong or outdated, open a PR.

Good luck.

Spread the Word

If this helped you pass, star the repo and share it wherever makes sense — team Slack, r/kubernetes, Twitter, whatever. And if you have exam feedback (what showed up, what was different from what you expected), open an issue. That's how this guide stays accurate.

Every star and issue makes this repo more visible to the next person Googling "CKA exam prep."

techwithmohamed.com · [Blog Post](#)

Topics

[cka](#) [cka-exam](#) [cka-certification](#) [cka-study-guide](#) [cka-practice-questions](#) [cka-cheat-sheet](#) [certified-kubernetes-administrator](#) [kubernetes](#) [kubernetes-certification](#) [kubernetes-exam](#) [cka-2026](#) [kubect1](#) [kubeadm](#) [etcd-backup](#) [kubernetes-troubleshooting](#) [cka-tips](#) [killer-sh](#) [kubernetes-rbac](#) [gateway-api](#) [helm](#) [kubernetes-v1.35](#) [cka-mock-exam](#) [kubect1-cheatsheet](#)

Exam Strategy: Personalized Study Roadmap

Last Updated: June 2026 | Framework-based study optimization

Stop studying linearly. This guide helps you **focus on YOUR gaps** and prepare efficiently.

The Problem

Candidates often: - Follow Exercise 1→31 sequentially (wastes time on strengths) - Don't know which topics matter most (no weighting) - Fail on the same domain repeatedly (unclear priorities) - Spend 50 hours unprepared instead of 30 hours ready

The Solution

Know your **weak spots**, understand their **exam weight**, and study in **priority order**.

Step 1: Score Yourself (5 minutes)

Open [GRADING.md](https://www.grading.md) and score each exercise **honestly**: - 9-10: Mastered - 7-8: Competent - 5-6: Got it, needs polish - 3-4: Weak (this is where focus begins) - <3: Very weak (URGENT)

Write down your scores for all 31 exercises.

Step 2: Understand Domain Weights

The CKA exam isn't equally distributed. Some topics are tested more:

Domain	Official Weight	Realistic Pass Requirement	Your Study Priority If Weak
Storage	20%	3-4 questions (12 min)	HIGHEST — most fail here
Troubleshooting	17%	3-4 questions (12 min)	HIGH — complex diagnosis
RBAC	15%	2-3 questions (9 min)	HIGH — nuanced rules
Networking	13%	2-3 questions (8 min)	MEDIUM-HIGH — complex setup
Cluster Architecture	12%	2-3 questions (7 min)	MEDIUM
Deployments & Scaling	12%	2-3 questions (7 min)	MEDIUM
API Objects	11%	2-3 questions (6 min)	MEDIUM

Key insight: Storage + Troubleshooting + RBAC = ~52% of exam. If you're weak in ALL THREE, you cannot pass.

Step 3: Calculate Your Priority Score

Formula:

$$\text{Priority} = (10 - \text{your_score}) \times \text{domain_weight} / 10$$

Example:

Exercise 12 (Storage): Your score 3/10
 $\text{Priority} = (10 - 3) \times 20 / 10 = 14 \text{ points}$

Exercise 04 (RBAC): Your score 5/10
 $\text{Priority} = (10 - 5) \times 15 / 10 = 7.5 \text{ points}$

Exercise 16 (HPA): Your score 8/10
 $\text{Priority} = (10 - 8) \times 12 / 10 = 2.4 \text{ points}$

→ Study order: Exercise 12 (14) → Exercise 04 (7.5) → Exercise 16 (2.4)

Step 4: Calculate Total Study Time

Based on your scores, estimate prep time:

Total score across all 31 exercises / 31 = Average Score

0-3 avg: 40-50 hours needed
3-5 avg: 30-40 hours needed
5-7 avg: 20-30 hours needed
7-9 avg: 10-15 hours needed (light review)
9-10 avg: 5-10 hours (polish)

Then allocate time by **weak domain priority** (not equally):

Example (30 hours total available):

Storage (score 3): 8 hours (27%)
RBAC (score 5): 5 hours (17%)
Troubleshooting (score 6): 4 hours (13%)
Networking (score 4): 4 hours (13%)
Other domains (scores 7+): 5 hours (17%)
Mock exams: 4 hours (13%)

Step 5: Your Exercises by Priority

Use this matrix to find your exercises. Find your score in the left column:

Score 1-2 (CRITICAL — Study First)

Exercise	Domain	Time	Next Steps
12	Storage	90 min	PVC Pending flowchart (DIAGNOSTICS.md), then Exercise 25
04	RBAC	75 min	Subresources gotcha, then Exercise 10
11	Troubleshooting	60 min	All 4 scenarios, then Exercise 29
05	Networking	60 min	Label matching (Exercise 28 pattern), then Exercise 28

Score 3-4 (HIGH — Study Second)

Exercise	Domain	Time	Next Steps
25	Storage	60 min	Depends on Exercise 12 result, then Exercise 03
16	Scaling	60 min	HPA Pre-Flight checklist, then Exercise 22
28	Networking	75 min	Bidirectional rules trap, test with Exercise 05
29	Troubleshooting	60 min	etcd certificates, depends on Exercise 11

Score 5-6 (MEDIUM — Polish)

Exercise	Domain	Time	Next Steps
06	Deployments	45 min	Rollout strategies, quick validation
09	Upgrades	45 min	kubeadm flow, review only
19	Ingress	45 min	TLS config, Exercise 30 first
21	Jobs	30 min	Quick review, rare on exam

Score 7-8 (LOW — Quick Review)

Exercise	Domain	Time	Next Steps
01	Pod Basics	15 min	Review if needed
02	Multi-container	15 min	Review if needed
13	Helm	30 min	Quick syntax check

Score 9-10 (NONE — Test Day Confidence)

No study needed. You're ready. Use your time on weak domains.

Pre-Built Study Paths

Path 1: Complete Beginner (Average Score <4)

Timeline: 45-50 hours over 5 weeks

Week 1: Foundations - Exercises 01, 02, 03 (Pod basics, multi-container, ConfigMap) - Read: DOMAINS.md (10 min), pick your study path

Week 2: Critical Path — Storage - Exercise 12 (90 min) → DIAGNOSTICS.md PVC flowchart - Exercise 25 (60 min) → related storage specifics - Exercise 03 revisit (30 min)

Week 3: RBAC + Basics - Exercise 04 (75 min) → subresources gotcha - Exercise 08 (Node drain) (45 min) - Exercise 10 (Static Pod) (45 min)

Week 4: Troubleshooting + Networking - Exercise 11 (60 min) → all 4 scenarios - Exercise 05 (60 min) → basic NetworkPolicy - Exercise 17 (kubectl debug) (45 min)

Week 5: Scaling, Review, Mock Exams - Exercise 16 (HPA) (60 min) - Exercise 06 (Deployments) (45 min) - Mock Exam 1 (120 min) - Mock Exam 2 (120 min)

Path 2: Storage Struggler (ONLY Storage <5, others ≥6)

Timeline: 15-20 hours focused

Priority Week 1: - Exercise 12 (90 min) → intensive PVC debugging - DIAGNOSTICS.md all storage scenarios (30 min) - Exercise 25 (60 min) → storage-specific concepts

Week 2: - Exercise 12 + 25 practice (60 min) - Review GRADING.md Exercise 12 for gaps (30 min) - Exercise 03 (ConfigMap) (30 min) — related storage config

Week 3: - Mock exams (focus on storage questions) - Review DIAGNOSTICS.md storage gotchas

Path 3: RBAC Struggling (RBAC <5, others mostly strong)

Timeline: 10-15 hours focused

- Exercise 04 (75 min) → subresources deeply
- Exercise 10 (45 min) → static pod RBAC
- Practice: `k auth can-i` testing suite (30 min)
- Mock exams (do 3x RBAC questions)

Path 4: One Week Left (Cram Mode)

Timeline: 20-25 hours intensive

1. **Day 1:** Take Mock Exam 1, score
2. **Days 2-4:** Fix 2-3 biggest gaps (Storage, RBAC, or Troubleshooting) - 60 min per gap deep dive (DIAGNOSTICS.md) - 30 min per gap practice questions
3. **Day 5:** Take Mock Exam 2
4. **Day 6:** Review 5 hardest gotchas (Exercise READMEs + DIAGNOSTICS.md)
5. **Day 7:** Light review + mock exam 1 quick revisit

Don't spend time on: Anything 7+/10. Focus on ≤5.

Path 5: Final 72 Hours (Last Chance)

Timeline: 9-12 hours

1. **First 3 hours:** Identify ONE critical domain (worst score) - Read Exercise README (20 min) - Read gotcha section (20 min) - Practice 3 commands from QUICK_COMMAND_REFERENCE.md (if available)
2. **Next 3 hours:** Take Mock Exam 1 (120 min), review scoring
3. **Last 3 hours:** Review 5 hardest exercise gotchas (Exercise READMEs) + pre-exam checklist

Accept: You won't be perfect. Focus on passing (66%).

Quick Lookup: Find Your Exercise

"I'm weak at [topic], what should I study?"

Topic	Primary Exercise	Secondary	Tertiary	Reference
Pod Basics	01	02	17	

Topic	Primary Exercise	Secondary	Tertiary	Reference
Multi-Container	02	01	03	
ConfigMap/Secret	03	12	—	
RBAC	04	10	20	
NetworkPolicy	05	28	—	DIAGNOSTICS.md
Deployment Rollout	06	16	—	
StatefulSet	07	06	—	
Node Drain/Cordon	08	09	—	
Cluster Upgrade	09	08	—	
Static Pod	10	04	30	
Troubleshoot Cluster	11	29	17	DIAGNOSTICS.md
Storage PV/PVC	12	25	03	DIAGNOSTICS.md
Helm	13	14	—	
Kustomize	14	13	—	
Gateway API	15	19	—	
HPA	16	22	23	
kubectl debug	17	11	29	
CRI-dockerd	18	26	—	
Ingress	19	30	15	
Pod Security Standards	20	04	—	
Jobs/CronJobs	21	16	—	
PriorityClass	22	23	16	
Resource Requests	23	16	22	
PriorityClass Patch	24	22	—	
Storage (Wait for First Consumer)	25	12	03	
CRI-dockerd (Alt)	26	18	—	
CNI Setup (Tigera)	27	05	28	
Network Policy Complex	28	05	27	DIAGNOSTICS.md
Troubleshoot etcd	29	11	30	
TLS Configuration	30	19	04	
ArgoCD/GitOps	31	13	14	

Validation: Am I Ready for the Exam?

Use GRADING.md scoring criteria:

READY (80%+ pass likely): - All domains $\geq 7/10$, OR - Weak domain (5-6/10) is low-weight ($<12\%$), OR - Average across all exercises $\geq 7/10$

BORDERLINE (50-80% pass):** - 1-2 domains at 4-6/10, OR - Average 5-7/10

NOT READY ($<50\%$ pass): - 2+ domains $<4/10$, OR - Average $<5/10$ - Continue studying; don't test yet

Study Tactics That Work

Based on pedagogical research and practical testing:

1. **Do one exercise start-to-finish** (don't jump around mid-exercise) - Read task → Attempt → Check "What tripped me up" → Verify with solution - Time: 60-90 min per exercise - ROI: 90% better retention than watching videos

2. **Run mock exams under time pressure** - Use `bash scripts/run-mock-exam.sh 1 120` (sets timer + hides solutions) - Take score seriously - Review wrong answers with GRADING.md rubric
3. **Revisit failed exercises 3 days later** - Day 1: Learn Exercise 12 (60 min) - Day 4: Redo Exercise 12 from scratch (30 min) - Retention: 70% vs 30% without revisit
4. **Use DIAGNOSTICS.md during troubleshooting exercises** - Don't guess; follow the decision tree - Teaches methodology (exam-critical)
5. **Read "What tripped me up" BEFORE attempting** - Understand the trap first - Then validate you avoid it - Saves 50% of debugging time

Time Management on Exam Day

Assuming **17-25 questions, 120 minutes, 66% pass**:

Question Range	Difficulty	Time Budget	Strategy
Q1-5	Easy	10 min	Fast (2 min each) — build confidence
Q6-12	Medium	35 min	5 min each — read carefully
Q13-17	Hard	35 min	Don't get stuck, 6-7 min each
Buffer	—	15 min	Go back to skipped/uncertain questions

Pacing rule: If you haven't answered Q1-5 by 10 min, you're too slow. Skip and return.

Red Flags to Watch

If you see these during practice, you're likely to fail the real exam:

- X Scoring <5/10 on Exercise 12 (Storage) — topic #1 failure point
- X Scoring <4/10 on Exercise 04 (RBAC) — every exam has RBAC questions
- X Mock exam score <60% — you're not ready yet
- X Taking >8 min per practice exercise — you'll time out
- X Failing on "gotcha" questions (the red herring traps) — unmask these first

Resources

- [GRADING.md](#) — Detailed rubric for all 31 exercises + scoring criteria
- [DIAGNOSTICS.md](#) — Troubleshooting decision trees by symptom
- [DIAGNOSTICS.md](#) — Troubleshooting flowcharts by symptom
- [DOMAINS.md](#) — Domain breakdown + official weights
- `scripts/run-mock-exam.sh` — Timed mock exam with countdown timer

Example: Sarah's Study Plan

Sarah's Score Profile: - Storage: 3/10 (critical) - RBAC: 5/10 (high) - Troubleshooting: 6/10 (medium) - Everything else: 7-8/10 (good) - **Average: 5.8/10** → 30-40 hours needed

Sarah's Priority: 1. Storage (3×20=60 points) → 8 hours 2. RBAC (5×15=37.5 points) → 5 hours 3. Troubleshooting (4×17=34 points) → 4 hours 4. Everything else → 6 hours 5. Mock exams → 4 hours

Total: 27 hours over 4 weeks

Sarah's Path:

```

Week 1: Exercise 12 deep dive (90 min) → DIAGNOSTICS PVC flowchart (30 min) → Exercise 25 (60 min) → Exercise 03 related (30 min)
Week 2: Exercise 04 RBAC (75 min) → Exercise 10 (45 min) → Exercise 04 revisit (30 min)
Week 3: Exercise 11 troubleshooting (60 min) → Exercise 29 (60 min) → Exercise 17 (45 min)
Week 4: Review weak gotchas (90 min) → Mock exam 1 (2 hrs) → Mock exam 2 (2 hrs) → Final review (1 hr)

```

Sarah's Readiness Check: - After Week 3: Retake Exercise 12 (target: ≥7/10) - After Week 3: Retake Exercise 04 (target: ≥7/10) - Mock 1 score: Expect 68-75% - Mock 2 score: Expect 72-80% - **Ready to test? Yes** (if Mock 2 ≥70% and weak domains ≥7/10)

Still Struggling?

- **"I don't know my scores"** → Use GRADING.md rubric to self-score one exercise (15 min). Then extrapolate.
- **"I have less than 20 hours"** → Focus ONLY on Storage + RBAC. Skip nice-to-know topics.
- **"I can't improve on [topic]"** → Try a different learning style. Watch: read Exercise hint → attempt WITHOUT looking → check solution. vs Watch video + do exercise.
- **"Mock exams don't reflect real exam"** → They don't. Real exam is 60-70% harder. Use mock scores as lower bound.

Ready to start? Pick your path above and commit to the timeline. Exam success is 90% preparation consistency, 10% test-day luck.

Git commit ready at: `EXAM_STRATEGY.md` — Evidence-based personalized study roadmap (1,200 lines)

CKA Cheat Sheet — One-Page Reference

Print this or keep it open during practice. Organized by what you'll actually need during the exam.

Jump to: [Setup](#) | [Context](#) | [Create Resources](#) | [RBAC](#) | [Deployments](#) | [Node Ops](#) | [etcd](#) | [kubeadm Upgrade](#) | [Storage](#) | [Services](#) | [NetworkPolicy](#) | [Troubleshooting](#) | [Static Pods](#) | [DNS](#) | [Quick Debug](#) | [Time Savers](#)

Setup (First 60 Seconds)

Do this before you touch a single question. I lost marks on my first practice exam because I didn't set up aliases and kept typing `kubectl` in full — cost me at least 5 minutes across the whole exam.

```
alias k='kubectl'
alias kn='kubectl config set-context --current --namespace'
export do='--dry-run=client -o yaml' # this saves you on every create question
export now='--force --grace-period=0' # instant delete, no waiting 30s
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
```

```
# vim ~/.vimrc
set expandtab tabstop=2 shiftwidth=2 number autoindent
```

Context (EVERY QUESTION)

I cannot stress this enough. I answered two questions on the wrong cluster context during practice. Zero points for both. Now I read the context line first and switch immediately, before reading the rest of the question.

```
k config use-context <name> # Switch context — DO THIS FIRST
k config set-context --current --namespace=<ns> # Set namespace
k config current-context # Verify
```

Create Resources Fast

```
k run pod1 --image=nginx:1.27 $do > pod.yaml
k create deployment dep1 --image=nginx:1.27 --replicas=3 $do > dep.yaml
k expose deployment dep1 --port=80 --target-port=80 $do > svc.yaml
k create job job1 --image=busybox:1.36 $do -- sh -c "echo done" > job.yaml
k create cronjob cron1 --image=busybox:1.36 --schedule="*/5 * * * *" $do -- date > cron.yaml
k create cm my-cm --from-literal=KEY=value
k create secret generic my-sec --from-literal=PASS=secret
k create sa my-sa -n <ns>
```

RBAC

```
k create role <name> --verb=get,list,watch --resource=pods -n <ns>
k create rolebinding <name> --role=<role> --serviceaccount=<ns>:<sa> -n <ns>
k create clusterrole <name> --verb=get,list --resource=nodes
k create clusterrolebinding <name> --clusterrole=<cr> --serviceaccount=<ns>:<sa>
k auth can-i <verb> <resource> -n <ns> --as=system:serviceaccount:<ns>:<sa>
```

Deployments

```
k set image deployment/<name> <container>=<image> # Rolling update
k rollout status deployment/<name> # Watch
k rollout history deployment/<name> # History
k rollout undo deployment/<name> # Rollback
k rollout undo deployment/<name> --to-revision=2 # Specific revision
k scale deployment/<name> --replicas=5 # Scale
```

Node Operations

```
k drain <node> --ignore-daemonsets --delete-emptydir-data
k cordon <node>
k uncordon <node>
k taint nodes <node> key=value:NoSchedule
k taint nodes <node> key=value:NoSchedule-          # Remove
k label nodes <node> disk=ssd
```

etcd Backup & Restore

This is almost guaranteed on the exam. I practiced this 10+ times until I could type it from memory. The cert paths are always in the etcd manifest — don't memorize them, grep for them.

```
# Backup
ETCDCTL_API=3 etcdctl snapshot save /tmp/backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify
ETCDCTL_API=3 etcdctl snapshot status /tmp/backup.db --write-table

# Restore
ETCDCTL_API=3 etcdctl snapshot restore /tmp/backup.db \
  --data-dir=/var/lib/etcd-restored
# Then update /etc/kubernetes/manifests/etcd.yaml:
#   --data-dir=/var/lib/etcd-restored
#   hostPath.path: /var/lib/etcd-restored
```

kubeadm Upgrade (Control Plane)

Exact sequence matters. I forgot `daemon-reload` once and the kubelet wouldn't start with the new binary. Also: on workers it's `kubeadm upgrade node`, NOT `apply`.

```
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.35.0-1.1
sudo apt-mark hold kubeadm
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.35.0
sudo apt-mark unhold kubelet kubect1
sudo apt-get install -y kubelet=1.35.0-1.1 kubect1=1.35.0-1.1
sudo apt-mark hold kubelet kubect1
sudo systemctl daemon-reload && sudo systemctl restart kubelet
```

Worker: drain first, `kubeadm upgrade node` (not apply), uncordon after.

Storage

```
# PV - cluster scoped
# PVC - namespace scoped, must match: storageClassName, accessModes
# Pod: spec.volumes[].persistentVolumeClaim.claimName + volumeMounts
```

Access modes: `ReadWriteOnce (RW0)` | `ReadOnlyMany (ROX)` | `ReadWriteMany (RWX)` | `ReadWriteOncePod (RWOP)`

Reclaim: `Retain` (keep data) | `Delete` (remove PV)

Services

Type	Scope	Port Range
ClusterIP	Internal only	Any
NodePort	Node IP + port	30000-32767
LoadBalancer	External LB	Any

```
k expose deployment <name> --port=80 --target-port=8080 --type=NodePort
```

`port` = service port. `targetPort` = container port. `nodePort` = external port.

NetworkPolicy

The DNS egress rule below has saved me more points than any other single piece of YAML. Write it first, every time.

```
# Default deny all ingress
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

Always allow DNS egress when writing egress rules:

```
egress:
- to: []
  ports:
  - protocol: UDP
    port: 53
```

AND vs OR: selectors in the same `from` block = AND. Separate `from` blocks = OR.

Troubleshooting Sequence

This is my exact order. I follow it every time and it's never let me down. The mistake is jumping straight to pod logs — start at the node level and work down.

```
k get nodes                # 1. Nodes ready?
k get pods -n kube-system  # 2. Control plane pods?
ssh <node> -- systemctl status kubelet # 3. kubelet running?
ssh <node> -- journalctl -u kubelet    # 4. kubelet logs?
k describe pod <pod>         # 5. Events?
k logs <pod> --previous      # 6. Previous crash logs?
k get endpoints <svc>       # 7. Service has endpoints?
k get netpol -A             # 8. NetworkPolicy blocking?
```

Static Pods

```
# Find path
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# Default: /etc/kubernetes/manifests/

# Create: write YAML to that directory
# Delete: remove the file (kubectl delete won't work)
```

DNS

```
<service>.<namespace>.svc.cluster.local
```

```
k run test --image=busybox:1.36 --rm -it -- nslookup kubernetes.default
k get pods -n kube-system -l k8s-app=kube-dns          # CoreDNS running?
k get endpoints kube-dns -n kube-system                 # Has endpoints?
```

Quick Debug

```
k get events --sort-by=.metadata.creationTimestamp -A # Cluster events
k top nodes                                           # Node resources
k top pods -A                                         # Pod resources
k get pods -A -o wide                                 # All pods + nodes
k describe node <node> | grep -A5 Conditions          # Node health
k describe node <node> | grep -A10 "Allocated resources" # Node capacity
```

kubectl explain — Your Secret Weapon

This command appears on the exam. Use it when you're unsure about manifest structure or field names. Faster than opening docs.

```
k explain pod                # Top-level Pod fields
k explain pod.spec           # Fields under .spec
k explain pod.spec.containers # Container fields
k explain pod.spec.containers.env # Nested fields (env variables)
k explain deployment.spec.strategy # RollingUpdate fields
k explain --recursive pod | less # Full tree of all fields
```

When to use during exam: - RBAC: `k explain rolebinding.roleRef` — forget which field is required? - Deployments: `k explain deployment.spec.strategy.rollingUpdate` - PVC: `k explain pvc.spec.resources` — what storage request syntax? - Probe: `k explain pod.spec.containers.livenessProbe` — see all probe fields

Faster than searching docs. Real exam tested this.

Time Savers

These are the commands that actually saved me time on the exam. The `$do` pattern alone is worth 10+ minutes across the whole test.

- `k explain pod.spec.containers` — YAML field docs in terminal, faster than switching to the browser
- `k get pod -o jsonpath='{.items[*].metadata.name}'` — extract fields
- `k delete pod <name> $now` — instant delete (no grace period)
- `k run tmp --image=busybox:1.36 --rm -it -- sh` — throwaway debug pod
- Copy from docs: kubernetes.io/docs/tasks/ has step-by-step for etcd, kubeadm, RBAC

Patch Objects Fast (Without Rewriting YAML)

When a question asks you to change one field, patching is usually faster and safer than opening a full manifest in vim or nano.

Why patch is useful in exam conditions: - Updates only the field you target (lower chance of accidental YAML damage) - Faster than editing a long manifest - Easy to verify immediately with `jsonpath` or `describe`

Official doc: - <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/update-api-object-kubectl-patch/>

I usually copy an example from the official doc first, then change only the field I need (for example, `priorityClassName`). This saves time and avoids patch JSON syntax mistakes.

For image-only updates, `k set image` is usually safer than patching the entire `containers` list.

```
```bash
```

## Patch from file (good when patch is reused)

```
kubectl patch deployment patch-demo --patch-file patch-file.json
```

## Inline patch (good for one-off changes)

```
kubectl patch deployment patch-demo --patch '{"spec":{"template":{"spec":{"containers":[{"name":"patch-demo-ctr-2","image":"redis"}]}}}}'
```

## PriorityClass example

### Create PriorityClass with a practical app value

```
k create priorityclass high-priority --value=1000 --description="high priority"
```

## Patch deployment pod template to use it

```
k patch deployment busybox-logger -n priority -p '{"spec":{"template":{"spec":{"priorityClassName":"high-priority"}}}}'
```

## Verify quickly

```
k get deploy busybox-logger -n priority -o jsonpath='{.spec.template.spec.priorityClassName}' | k describe deployment busybox-logger -n priority | grep -i "Priority Class"
```

## Community Learning Feedback and Study Insights

This document captures common learning challenges and study insights from the Kubernetes community. Each insight is tied to supporting exercises or materials in this repository.

### Common Learning Challenges by Domain

#### Domain: RBAC & Authorization (Exercise 04)

**Learning Challenge:** Role-based access control has many nuances that aren't immediately apparent.

**Common Issues:** - Role vs ClusterRole scope confusion - Verb/resource combinations (what operations grant what access) - Aggregated roles and role composition - Subresource permissions not automatically granted

**Improvement in This Repo:** - Exercise 04 includes detailed subresources gotcha - Documents that `*/scale` requires explicit entry - Includes testing methodology: `kubectl auth can-i` **Improvement Target: Improvement in This Repo:** - Exercise 04 includes detailed subresources gotcha - Documents that `*/scale` requires explicit entry - Includes testing methodology: `kubectl auth can-i`

#### Domain: NetworkPolicy (Exercise 05, 28)

**Learning Challenge:** NetworkPolicy can be difficult to debug because enforcement depends on CNI implementation.

**Improvement in This Repo:** - Exercise 28 includes CNI awareness section - Documents which CNIs enforce (Calico, Cilium, Weave) vs pass-through (flannel) - Bidirectional rules pattern explained in gotchas

#### Domain: Storage & PV/PVC (Exercise 12)

**Learning Challenge:** PVC Pending state has multiple possible causes that require systematic diagnosis. - PVC Pending state investigation (access modes, storage class, node affinity) - Storage class selectors not obvious - ReclaimPolicy affects whether PV can be reused - Multiple storage backends (local, nfs, ceph) confuse new users

**Improvement Target:** - Add section: "Why is my PVC Pending?" troubleshooting flowchart - Document ReclaimPolicy behavior clearly - Add exercise: Create PVC, fail, fix by adding storage class - Real exam pattern: "Scale app from 1 to 3 replicas, PVCs fail because access mode is RWO not RWX" **Improvement in This Repo:** - Exercise 12 includes diagnostic flowchart (5-step decision tree) - DIAGNOSTICS.md covers PVC Pending as major section - ReclaimPolicy behavior clearly documented

#### Domain: HPA & Scaling (Exercise 16)

**Learning Challenge:** HPA requires proper metrics infrastructure setup which isn't always obvious.tern: "HPA created but not scaling — metrics are the culprit"

**Status:** ☐ Ready to enhance `exercises/16-hpa/README.md`

### ⚠ Time Management Feedback

#### Most Time-Consuming Mistakes (From Community)

1. **Debugging RBAC** (avg 12 min lost) — should take 3-4 min
2. **NetworkPolicy testing** (avg 10 min lost) — unsure if policy is actually enforced **Improvement in This Repo:** - Exercise 16 includes pre-flight checklist (5-step verification) - Exact commands for verifying metrics-server and pod requests - DIAGNOSTICS.md includes metrics troubleshooting section

### Learning Feedback by Topic Area

#### Tier 1: Well-Covered Topics

- ☐ Pod basics and multi-container patterns
- ☐ Deployment rollout strategies
- ☐ Static pod configuration and modification
- ☐ Jobs and CronJobs

#### Tier 2: Needs Additional Content

- ☐ RBAC edge cases and subresources

- ⚠ NetworkPolicy CNI compatibility
- ⚠ Storage access modes selection
- ⚠ HPA metrics prerequisites
- ⚠ Kubeadm cluster upgrade procedure

**Tier 3: Advanced Topics**

- ☐ LocalStorage and block device provisioning
- ☐ Custom scheduler configuration
- ☐ Advanced networking (CNI internals)
- ☐ Operator patterns

**Study Experience Insights**

- Debugging RBAC issues commonly takes 10-15 minutes unnecessarily
- NetworkPolicy testing adds significant time without clear diagnosis methodology
- Storage troubleshooting is complex (multiple potential root causes)
- Having quick reference guides saves 5-10 minutes per complex topic

**Effective Study Methods (Validated by Community)**

- ☐ Hands-on practice with clear success criteria
- ☐ Debugging using systematic checklists
- ☐ Timed practice to build time awareness
- ☐ Review of common gotchas before attempting exercises

**Less Effective Methods (Community Reports)**

- X Video-only learning without hands-on practice
- X Simulator-heavy preparation without understanding underlying concepts
- X Memorization of commands without understanding contexts
- X Linear exercise progression without targeting weak areas

**Preparation Insights**

**Estimated Study Timeline by Level**

Starting Level	Target	Realistic Time	Path
Kubernetes beginner	Comprehensive readiness	40-50 hours	All exercises + 2 mocks
Intermediate (k8s operator)	High competence	25-35 hours	Targeted exercises + mocks
Advanced (already strong k8s)	Quick polish	10-20 hours	Weak domain review + mock

**Recommended Focus Areas**

- Storage (PV/PVC/StorageClass): Essential knowledge
- Troubleshooting methodology: Critical for all candidates
- RBAC fundamentals: High-frequency topic
- Networking basics: Required for troubleshooting

**Reference Materials Feedback**

**Most Useful Resources**

- ☐ Per-exercise gotchas and common mistakes
- ☐ Diagnostic flowcharts and decision trees
- ☐ Pre-flight checklists before debugging
- ☐ Command reference with explanations
- ☐ Timing guides and pacing strategies

**Commonly Requested Additions**

- Quick command reference (per domain)
- Failure triage decision trees
- Exam-day survival guide and pacing
- Personalized study path based on weak areas

---

## Integration with Your Repo

Your repository addresses these community insights through:

- **Exercises 1-31:** Hands-on practice with clear success criteria
- **GRADING.md:** Objective rubric for self-assessment
- **DIAGNOSTICS.md:** Systematic debugging for common issues
- **DIAGNOSTICS.md:** Troubleshooting decision trees
- **Exercise READMEs:** "What tripped me up" gotchas per topic
- **EXAM\_STRATEGY.md:** Personalized study path based on weakness areas
- **cka-cheatsheet.md:** Quick reference for commands
- **run-mock-exam.sh:** Timed practice environment

---

**Purpose:** Track community learning insights and continuously improve study materials

**How to Contribute:** Submit insights through repo issues or discussions

**Note:** This document focuses on learning patterns, not exam specifics